

РЕФЕРАТ

Выпускная квалификационная работа, 97 страниц, 19 рисунков, 18 таблиц, 27 источников, 2 приложения.

Ключевые слова: система позиционного отслеживания, транспортные средства, реальное время, геозоны, пространственно-временной поиск, телеметрия, микросервисы, Apache Kafka, Google S2.

Актуальность: В связи с массовым оснащением транспортных средств аппаратурой спутниковой навигации и требованиями законодательства РФ растёт потребность в высокопроизводительных системах мониторинга. Существующие аналоги не обеспечивают одновременно высокой производительности зонального отслеживания в реальном времени, пространственно-временного поиска телеметрии и горизонтальной масштабируемости.

Цель работы: спроектировать и реализовать систему позиционного отслеживания транспортных средств с обработкой событий в реальном времени.

В процессе использовались программные средства: Open IDE, Visual Studio Code, Docker, Docker Compose, Git, GitHub Actions, Postman, Bruno, Kover.

Примененные языки программирования: Kotlin, TypeScript, SQL.

Полученные результаты: разработана микросервисная система, включающая приёмник телеметрии, брокер сообщений Apache Kafka, модули зонального отслеживания (на основе пространственного индекса S2) и пространственно-временного поиска, а также веб-интерфейс на Vue.js. Система выдерживает нагрузку более 2000 сообщений в секунду, задержка генерации событий не превышает 500 мс в 95% случаев. Модульное тестирование обеспечивает покрытие кода >90%.

Область применения: транспортные предприятия (грузовые, пассажирские, специальные перевозки), логистические компании, диспетчерские службы, требующие эффективного мониторинга автопарка в реальном времени.

ABSTRACT

Final qualifying work, 97 pages, 19 figures, 18 tables, 27 sources, 2 appendices.

Keywords: position tracking system, vehicles, real time, geo-fences, spatio-temporal search, telemetry, microservices, Apache Kafka, Google S2.

Relevance: Due to the massive equipping of vehicles with satellite navigation equipment and the requirements of the Russian legislation, the need for high-performance monitoring systems is growing. Existing analogues do not simultaneously provide high performance of real-time geofencing, spatio-temporal telemetry search and horizontal scalability.

Purpose of the work: to design and implement a real-time vehicle position tracking system with event processing.

Software tools used in the process: OpenIDE, Visual Studio Code, Docker, Docker Compose, Git, GitHub Actions, Postman, Bruno, Kover.

Programming languages used: Kotlin, TypeScript, SQL.

Results obtained: a microservice system has been developed, including an telemetry receiver, an Apache Kafka message broker, geofencing modules (based on the S2 spatial index) and a spatio-temporal search module, as well as a Vue.js web interface. The system withstands a load of more than 2000 messages per second, the event generation delay does not exceed 500 ms in 95% of cases. Unit testing provides code coverage >90%.

Application area: transport enterprises (freight, passenger, special transportation), logistics companies, dispatching services requiring efficient real-time fleet monitoring.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	8
1 Анализ предметной области	11
1.1 Описание предметной области	11
1.1.1 Современное состояние предметной области.....	11
1.1.2 Методы и алгоритмы решения задач	12
1.2 Постановка задачи.....	14
1.2.1 Функциональные требования	14
1.2.2 Нефункциональные требования	16
1.3 Сравнение с аналогами.....	17
1.3.1 Обзор существующих аналогов.....	18
1.3.2 Сравнительная таблица характеристик	20
1.3.3 Обоснование целесообразности собственной разработки.....	22
2 Разработка системы.....	23
2.1 Выбор платформы для реализации	23
2.1.1 Операционная система и аппаратное обеспечение	23
2.1.2 Язык программирования и серверные фреймворки.....	24
2.1.3 Системы управления базами данных и кэширования	24
2.1.4 Выбор используемых библиотек	25
2.1.5 Платформа клиентского веб-интерфейса	25
2.1.6 Инфраструктурные решения.....	26
2.2 Выбор средств разработки	26
2.2.1 Система контроля версий.....	26
2.2.2 Интегрированные среды разработки.....	26

2.2.3 Система сборки и управления зависимостями	27
2.2.4 Инструменты тестирования	27
3 Реализация и полученные результаты	28
3.1 Описание разработки программного обеспечения	28
3.1.1 Проектирование архитектуры системы	28
3.1.2 Описание модулей системы	29
3.1.3 Пространственный индекс	35
3.2 Описание структур данных	42
3.2.1 Диаграмма отношений сущностей	42
3.2.2 Словарь данных	43
3.3 Описание разработки интерфейса пользователя	49
3.3.1 Интерфейс ведения справочника объектов	49
3.3.2 Интерфейс мониторинга транспортных средств	52
4 Тестирование и ввод в эксплуатацию	55
4.1 Тестирование программного обеспечения	55
4.1.1 Автоматизированное модульное тестирование	56
4.1.2 Ручное функциональное тестирование	58
4.1.3 Нагрузочное тестирование	61
4.2 Ввод в эксплуатацию	63
ЗАКЛЮЧЕНИЕ	68
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	70
ПРИЛОЖЕНИЕ А Системные требования	74
ПРИЛОЖЕНИЕ Б Исходный код	75

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API — Application Programming Interface (программный интерфейс приложения).

CI/CD — Continuous Integration / Continuous Delivery (непрерывная интеграция и непрерывная доставка).

CPU — Central Processing Unit (центральный процессор, ЦП).

GiST — Generalized Search Tree (обобщённое поисковое дерево; тип индекса в PostgreSQL).

HTTP — HyperText Transfer Protocol (протокол передачи гипертекста).

JSON — JavaScript Object Notation (текстовый формат обмена данными).

SQL — Structured Query Language (язык структурированных запросов).

СУБД — система управления базами данных.

WebSocket — протокол полнодуплексной связи по TCP.

Геохеширование — метод представления географических координат или областей в виде дискретных ячеек/интервалов.

S2 (Google S2 Geometry Library) — библиотека геохеширования и сферической дискретизации, применяемая для построения пространственных индексов.

Геозона — географическая область, заданная многоугольником или окружностью, используемая для контроля местоположения ТС.

Телеметрия (телеметрические данные) — совокупность навигационных и диагностических параметров, передаваемых бортовым устройством ТС.

Docker — платформа для контейнеризации приложений.

ВВЕДЕНИЕ

Сфера транспортных перевозок является важной частью экономики современного мира и, в частности, Российской Федерации. Посредством различных типов транспорта, таких как железнодорожный транспорт, воздушный транспорт, автомобильный транспорт, водный транспорт в Российской Федерации перевозится не менее 7 миллиардов тонн грузов ежегодно за период с 2000 по 2024 год. Объем пассажирских перевозок за тот же период составляет ежегодно не менее 12,4 миллиардов человек [1].

Большой объем транспортной работы в условиях повсеместной цифровизации ставит перед транспортными предприятиями задачи сбора, хранения, обработки и передачи большого объема информации, связанного с транспортной деятельностью.

Например, с 1 сентября 2021 года все транспортные средства категорий М2, М3 и N в обязательном порядке должны быть оборудованы аппаратурой спутниковой навигации, выполняющей передачу данных в Ространснадзор через оператора системы «ЭРА-ГЛОНАСС» [2]. Согласно постановлению Правительства РФ №504 от 14.06.2013 все автомобильные транспортные средства, имеющие разрешенную максимальную массу более 12 тонн, обязаны передавать навигационные данные для автоматического расчета оплаты за пользование дорожной инфраструктурой [3].

Указанные выше факторы привели к появлению и активному развитию на протяжении десятилетий систем отслеживания транспортных средств, призванных решить различные задачи, стоящие перед транспортными предприятиями.

В качестве задач, являющихся общими для подобных систем, можно указать следующие: прием сигналов оборудования, обработка (формирование) телематической информации, хранение и анализ телематической информации. Эти задачи являются общими, поскольку обеспечивают доступность данных, используемых для решения других более узких задач.

Также в качестве общих можно указать следующие задачи: зональное отслеживание транспортных средств, пространственно-временной поиск телеметрии. Зональное отслеживание позволяет контролировать местоположение ТС. Пространственно-временной поиск позволяет выполнять поиск телеметрической информации по географической области и временному интервалу.

Таким образом, становится понятен образ базового перечня потребностей транспортного предприятия любой направленности (грузоперевозки, пассажирские перевозки и т.д.), а соответственно, и требований к системам отслеживания транспорта и задачам информатизации, которые должны быть решены в ходе проектирования такой системы.

Рассмотрим подходы, которые могут быть применены в ходе решения ранее перечисленных задач.

Задачи приема, формирования и хранения телеметрической информации должны решаться в соответствии с типом использованного оборудования и типом протокола, который используется при приеме данных. Наиболее интересными проблемами, связанными с решением этих задач, являются задачи повышения производительности сервисов, выполняющих обработку телеметрических данных. Наиболее эффективными выглядят решения, относящиеся к архитектуре программной системы.

Задача зонального отслеживания транспортных средств сводится к задаче определения вхождения точки на поверхности земного шара (местоположение ТС) в ограниченную географическую область. Для определения вхождения могут быть применены следующие методы: перебор зон с проверкой вхождения, применение пространственных индексов в различных СУБД, использование иерархических пространственных деревьев, использование систем геохеширования (geohash, Google S2, Uber H3) для создания пространственных индексов в оперативной памяти. Наиболее оптимальным представляется подход с использованием геохеширования.

Пространственно-временной поиск данных позволяет запрашивать данные, ограничивая запрос по времени поступления информации и по географической области поступления информации. Данные телеметрии хранятся в какой-либо СУБД. На относительно небольших объемах данных и при небольшой нагрузке на базу данных возможно использование только временного индексирования с проверкой вхождения в географическую область на уровне приложения. Второй подход - комбинация двух отдельных индексов – пространственного и по времени – в тех СУБД, которые поддерживают подобный подход. Наиболее оптимальным является применение геохеширования (кодирования) географической позиции телеметрических данных и использования составного индекса по результату кодирования и по времени.

Цель работы – спроектировать и реализовать систему позиционного отслеживания транспортных средств с обработкой событий в реальном времени.

Для достижения цели решаются задачи:

- проанализировать предметную область, её современное состояние, обосновать актуальность разработки;
- выбрать и описать стек технологий, обосновать выбор;
- реализовать программный продукт, описать полученные результаты, провести тестирование разработанного решения.

Объект исследования – процесс обеспечения обработки информации в системах позиционного отслеживания транспортных средств.

Предмет исследования – программный продукт, выполняющий функции позиционного отслеживания транспортных средств с обработкой событий в реальном времени.

Практическая ценность работы заключается в повышении эффективности использования ресурсов, снижении эксплуатационных затрат и увеличении устойчивости систем отслеживания транспорта в условиях высоких нагрузок.

1 Анализ предметной области

1.1 Описание предметной области

Описание предметной области охватывает современное состояние методов решения проблемы с указанием современной литературы, приводится описание возможных способов её решения, их достоинства и недостатки.

1.1.1 Современное состояние предметной области

Современное транспортное предприятие, независимо от его профиля (грузовые, пассажирские, специальные перевозки), функционирует в условиях жесткой конкуренции и возрастающих требований со стороны государства. Подобная среда располагает к формированию потока телематической информации, который должен быть принят, обработан и сохранен для последующего анализа.

Транспортные предприятия решают специфические внутренние производственные задачи, что обуславливает функциональный состав систем отслеживания. Анализ существующих решений и публикаций [4–5] позволяет выделить следующий базовый перечень задач, решаемых такими системами:

1. Приём, обработка и хранение телеметрических данных, поступающих от бортового оборудования различных производителей с использованием различных протоколов.
2. Зональное отслеживание – контроль вхождения транспортного средства в заданные географические области (зоны) и генерация соответствующих событий в режиме, близком к реальному времени.
3. Пространственно-временной поиск телеметрии – выборка данных о местоположении и параметрах движения транспортных средств с одновременными ограничениями по географическому региону и временному интервалу.

Каждая из перечисленных задач обладает собственной спецификой, порождающей требования к алгоритмическим и архитектурным решениям.

1.1.2 Методы и алгоритмы решения задач

1. Приём, обработка и хранение телеметрии.

Поток навигационных данных от тысяч транспортных средств носит асинхронный и высокоинтенсивный характер. Традиционный монолитный подход к построению подсистемы приёма данных плохо масштабируется и не обеспечивает требуемой отказоустойчивости [5][6]. В современной литературе в качестве стандарта для решения подобных задач рассматриваются архитектуры, основанные на микросервисах и брокерах сообщений (Apache Kafka, RabbitMQ) и разделении хранилищ на оперативные (базы данных в ОЗУ, кэши) и аналитические (реляционные СУБД, колоночные хранилища) [7]. Такое разделение позволяет независимо масштабировать компоненты приёма, первичной обработки и долговременного хранения, а также изолировать сбои, предотвращая отказы.

2. Зональное отслеживание.

Задача определения принадлежности точки с географическими координатами заданной области является классической задачей вычислительной геометрии. Для многоугольных зон применяются алгоритмы, такие как метод трассировки луча [8]. Однако при необходимости обработки миллионов сообщений и тысяч зон прямой перебор зон с полной геометрической проверкой неприемлем из-за вычислительной сложности $O(K \cdot N)$, где K – число зон, N – число точек.

Для снижения сложности используют методы пространственного индексирования:

- Пространственные индексы в СУБД (например, GiST в PostGIS). Такой подход эффективен для пакетной и аналитической обработки, но при использовании потоковой архитектуры вызывают задержки, связанные с обращениями к дисковой подсистеме [9].

- Иерархические пространственные деревья. Обеспечивают логарифмическую сложность поиска кандидатов, однако в худших случаях

(неравномерное распределение, вытянутые полигоны) могут требовать проверки большого числа кандидатов и сложны в балансировке [10].

– Системы геохеширования и сферической дискретизации, самыми популярными представителями которых можно назвать geohash, Google S2, Uber H3. Эти методы набирают популярность благодаря возможности представить географическую область в виде набора дискретных ячеек и выполнять проверку вхождения точечного объекта за логарифмическое количество операций бинарного поиска по предварительно рассчитанному интервальному индексу, хранимому в оперативной памяти. В частности, библиотека Google S2 позволяет строить компактные индексы, сохраняющие предсказуемую производительность при любом количестве и форме зон, что критически важно для обработки событий в реальном времени [11, 12].

Таким образом, для систем с жёсткими временными ограничениями наиболее перспективным представляется использование пространственных индексов на основе системы геохеширования S2, обеспечивающих высокую скорость проверки вхождения ценой предварительного разложения зон на наборы ячеек заданного уровня.

3. Пространственно-временной поиск.

Запросы, объединяющие пространственные и временные фильтры, являются типовым инструментом диспетчерского анализа. Один из наиболее эффективных подходов заключается в создании составных индексов в реляционных СУБД (например, B-tree индекс по столбцам sector_id, timestamp), где идентификатор сектора получают заранее определённой пространственной дискретизацией [11]. Альтернативой выступают специализированные time-series базы данных с поддержкой геопространственных типов (TimescaleDB + PostGIS), которые позволяют эффективно обрабатывать подобные запросы за счёт партиционирования по времени и использования пространственных индексов внутри партиций [9]. Также возможен вариант с использованием двух не связанных между собой индексов в тех СУБД, которые поддерживают

построение плана по разным индексам для одного запроса [13]. Наиболее производительным и гибким решением представляется составной индекс в СУБД по времени и идентификатору сектора, поскольку мощность такого индекса легко контролировать изменением уровня геохеширования, нет необходимости партиционирования, которое уменьшает эффективность запросов, не задействующих ключ партиционирования, а также не зависит от планировщика запросов так же сильно, как подход с двумя отдельными индексами.

1.2 Постановка задачи

На основании проведённого анализа предметной области, существующих систем-аналогов и современных технологических подходов сформулированы цель и требования к разрабатываемой системе позиционного отслеживания транспортных средств с обработкой событий в реальном времени.

Цель разработки – создание высокопроизводительной, масштабируемой и отказоустойчивой программной платформы, обеспечивающей приём, обработку, хранение телеметрической информации, зональное отслеживание транспортных средств в режиме реального времени, пространственно-временной поиск телеметрии.

1.2.1 Функциональные требования

Система должна обеспечивать выполнение следующих основных функций:

1. Приём телеметрических данных:

- приём навигационных и диагностических пакетов от бортового оборудования;
- потоковая обработка входящих сообщений с гарантией сохранения порядка в рамках одного транспортного средства;
- подтверждение приема данных только после подтверждения их отправки во внутренний брокер сообщений.

2. Первичная обработка и хранение телеметрии:

- разбор входящих данных (формирование внутренних телеметрических пакетов);

- сохранение данных в СУБД для дальнейших запросов.

3. Зональное отслеживание в реальном времени:

- ведение реестра геозон, описываемых многоугольниками и окружностями;

- автоматическое определение факта вхождения/выхода транспортного средства в/из контролируемой зоны в момент поступления телеметрического пакета;

- генерация событий (вход, выход) и их доставка заинтересованным подсистемам;

- обеспечение отслеживания в реальном времени при количестве зон 50 000 в одном модуле отслеживания при условии соответствия системы системным требованиям. Количество хранимых в справочнике геозон должно быть ограничено 1 000 000 зонами, которое на практике в большинстве случаев недостижимо. Возможность обработки большего числа зон, чем 50 000 в реальном времени должна быть обеспечена возможностью горизонтального масштабирования и наращивания ресурсов системы.

4. Пространственно-временной поиск телеметрии:

- выборка телеметрических записей по комбинированному фильтру: географический регион (прямоугольник) и временной интервал;

- использование дискретизированных пространственно-временных индексов (составной индекс по идентификатору ячейки (геохеш) и времени) для эффективного выполнения запросов;

- предоставление пользовательского интерфейса для отображения треков за запрошенный период на карте, вывод списка транспортных средств,

находившихся за указанный временной интервал в запрошенной географической области.

5. Предоставление интерфейсов:

- веб-интерфейс для администраторов и диспетчеров, обеспечивающий визуализацию текущего положения транспортных средств на карте, историю треков, конфигурирование геозон и просмотр событий.

В ходе работы система оперирует описанными далее данными.

Входные данные: телеметрические пакеты, содержащие идентификатор устройства, географические координаты, скорость, направление, высоту, показания датчиков, временную метку; пакеты поступают от бортовых навигационных устройств.

Выходные данные:

- события телеметрического контроля (тип события, объект, время, координаты, метаданные)
- события зонального контроля (тип события, объект, зона, время);
- результаты пространственно-временных запросов (набор телеметрических записей).

Хранение данных: долговременное хранение всех необработанных и обработанных телеметрических записей в реляционной СУБД; хранение конфигураций (геозоны, правила) в реляционной СУБД, кратковременное хранение данных в персистентном брокере сообщений (Apache Kafka).

1.2.2 Нефункциональные требования

1. Производительность:

- система должна обрабатывать поток не менее 2 000 сообщений в секунду на рекомендуемом аппаратном обеспечении (Приложение А);
- задержка между поступлением пакета в приемную службу и генерацией события входа/выхода из зоны не должна превышать 500 мс в 95% случаев при нагрузке 2 000 сообщений в секунду.

2. Масштабируемость:

- архитектура должна позволять горизонтальное масштабирование компонентов приёма, обработки и хранения данных;

- должен быть использован микросервисный подход и применено асинхронное взаимодействие через брокер сообщений для независимого расширения узких мест.

3. Отказоустойчивость:

- система должна сохранять работоспособность при временной недоступности внешних сервисов (очереди повторной отправки);

- предусмотреть аппаратное разделение критичных компонентов для устранения единой точки отказа и уменьшении вероятности полной потери работоспособности;

- автоматическое восстановление после сбоев без потери уже принятых данных.

4. Безопасность:

- аутентификация и авторизация пользователей веб-интерфейса и API;

- межсервисное взаимодействие по защищённым каналам (SSL/TLS);

- журналирование действий пользователей и административных операций.

5. Сопровождение и документация:

- код должен сопровождаться технической документацией и руководством по развёртыванию;

- должны быть предусмотрены средства мониторинга работоспособности (метрики поступления, задержек, ошибок обработки).

1.3 Сравнение с аналогами

Для оценки конкурентоспособности разрабатываемой системы позиционного отслеживания транспортных средств с обработкой событий в реальном времени был проведён сравнительный анализ наиболее

распространённых на рынке решений. Основное внимание уделялось функциональным возможностям, производительности геозонального отслеживания, наличию пространственно-временного поиска, масштабируемости и отказоустойчивости.

1.3.1 Обзор существующих аналогов

1. Traccar

Открытая платформа спутникового мониторинга (лицензия Apache 2.0), поддерживающая широкий спектр протоколов телематического оборудования. Функциональность включает отображение текущего положения, историю треков, отчёты и базовое зональное отслеживание.

Преимущества: бесплатное распространение, большое сообщество, простой REST API, есть поддержка множества протоколов. Присутствует широкая поддержка ретрансляции данных: как «сырых», так и предварительно обработанных.

Недостатки: геозональное отслеживание, хоть и не приводит к одиночным запросам при каждой проверке, используя кеширование, тем не менее, не использует пространственный индекс, ввиду чего при достижении большого количества геозон скорость обработки будет сильно падать [18]. Также отсутствует встроенная возможность пространственно-временного поиска телеметрических данных.

Пространственно-временной поиск отсутствует.

2. Wialon

Проприетарная платформа, один из лидеров рынка. Предоставляет развитый интерфейс, сотни типов отчётов, продвинутое зональное отслеживание и интеграцию с датчиками. Возможно развертывание в виде локальной версии (на собственном оборудовании предприятия).

Преимущества: высокая надёжность (заявлено время доступности 99,98 %), регулярные обновления, поддержка большинства оборудования, мощная аналитика.

Недостатки: высокая стоимость лицензий, особенно для крупных автопарков; закрытый исходный код ограничивает возможности кастомизации. С 1 января 2026 года все операции на территории РФ прекращены.

Количество геозон и других элементов контроля ограничено количеством 31744, чего может быть недостаточно для крупного транспортного предприятия [19]. Из-за закрытости платформы и ограничения количества геозон невозможно достоверно выяснить, используются ли алгоритмы пространственной индексации. При заявленном трафике 944 точек в секунду и ограничения количества геозон гарантированная вычислительная сложность может быть значительно превышена с использованием подходов, предлагаемых в данной работе.

Пространственно-временной поиск отсутствует.

3. Navixy

Коммерческая платформа. Обладает широкими возможностями интеграций с различным оборудованием. Имеет локальную версию. В РФ продается в локализованном виде под названием «ГМ Телематика».

Преимущества: гибкая система отчётов, удобный конструктор дашбордов, поддержка различных протоколов.

Недостатки: для получения конкурентоспособной производительности геозон требуется горизонтальное масштабирование, которое в локальной версии ограничено; зависимость от вендора.

Пространственно-временной поиск отсутствует.

4. 1С:Центр спутникового мониторинга ГЛОНАСС/GPS

Российское решение, интегрированное с платформой 1С:Предприятие. Позволяет организовать мониторинг транспорта в единой информационной среде предприятия, поддерживает работу с оборудованием, передающим данные в систему ЭРА-ГЛОНАСС.

Преимущества: тесная интеграция с учётными системами 1С, соответствие требованиям российского законодательства. Есть события отслеживания в реальном времени.

Недостатки: временная сложность обработки зонального отслеживания линейная, пространственная индексация отсутствует. Уже на количестве около 200 ТС обработка входов/выходов в геозоны может занимать значительное количество времени. Пространственно-временной поиск отсутствует. Технических ограничений на количество ТС и зон нет, но лицензия приобретается на определенное количество объектов мониторинга, ввиду чего обслуживание крупного автопарка с помощью 1С может стать невыгодным, кроме того, на большом количестве объектов мониторинга скорость работы и количество затрат на вычислительные ресурсы могут стать неудовлетворительными.

5. Fort Monitor

Отечественная система мониторинга транспорта, сертифицированная для использования в государственных структурах. Поддерживает наиболее популярные протоколы, используемые в РФ.

Преимущества: полное соответствие российским нормативным требованиям, развитая картографическая подсистема, собственное серверное решение.

Недостатки: закрытость архитектуры; гарантированная скорость работы системы (в среднем 50 сообщений в секунду) с учетом задекларированных системных требований не позволяет сделать вывод о наличии высокоэффективной пространственной обработки [20]. Пространственно-временной поиск отсутствует. Возможность горизонтального масштабирования не описана.

1.3.2 Сравнительная таблица характеристик

В таблице 1.1 приведено сравнение ключевых характеристик рассмотренных аналогов с показателями разрабатываемой системы.

Таблица 1.1 — Сравнение аналогов

Характеристика	Разрабатываемая система (целевая архитектура)	Traccar	Wialon	Navixy	1С:ЦСМ	Fort Monitor
Производительность зонального отслеживания	Очень высокая	Низкая	Высокая	Высокая	Низкая	Средняя
Пространственно-временной поиск	Да	Нет	Нет	Нет	Нет	Нет
Масштабируемость	Горизонтальная	Горизонтальная	Вертикальная	Горизонтальная /Вертикальная	Вертикальная	Вертикальная
Открытость кода	-	Открытый	Закрытый	Закрытый	Закрытый	Закрытый
Лицензионная стоимость	-	Бесплатно	Высокая	Средняя/высокая	Средняя	Средняя
Доступность в РФ	Да	Да	Нет	Локализация	Да	Да

1.3.3 Обоснование целесообразности собственной разработки

Анализ показывает, что ни одно из существующих решений не обеспечивает одновременно:

- высокой производительности зонального отслеживания в режиме реального времени;
- пространственно-временного поиска телеметрических данных;
- возможности горизонтального масштабирования.

Таким образом, разработка собственной системы позиционного отслеживания с заявленными характеристиками является обоснованной. Предлагаемое решение позволит транспортным предприятиям повысить эффективность управления парком, сократить расходы на инфраструктуру, получить новые аналитические возможности, приобрести продукт, который может быть использован даже при значительном увеличении количества объектов мониторинга.

2 Разработка системы

2.1 Выбор платформы для реализации

Выбор технологического стека является важным этапом проектирования системы позиционного отслеживания транспортных средств, поскольку он напрямую определяет достижимость предъявленных к системе нефункциональных требований: производительности, масштабируемости, отказоустойчивости и безопасности.

2.1.1 Операционная система и аппаратное обеспечение

В качестве целевой операционной системы для серверной части выбрана ОС семейства Linux Ubuntu 24.04 LTS. Этот выбор продиктован следующими факторами:

- наилучшей поддержкой контейнеризации и инструментов оркестрации (Docker, Kubernetes) [21];
- популярностью дистрибутивов Ubuntu, развитое сообщество;
- преимуществами выпуска LTS (длительный период выпуска системных обновлений и патчей безопасности);
- бесплатность операционной системы для любого вида использования.

Аппаратная конфигурация целевого сервера определяется необходимостью обслуживать входящий поток не менее 2000 телеметрических сообщений в секунду. Исходя из практики развёртывания аналогичных систем, достаточными являются:

- CPU: 8 физических ядер с тактовой частотой не ниже 3.0 ГГц;
- оперативная память: 16 ГБ (для размещения пространственного индекса, кэша Redis и рабочих процессов сервисов);
- дисковая подсистема: SSD-накопитель объёмом не менее 500 ГБ (под оперативное хранение очередей Kafka), а также накопитель объёмом от 1 ТБ для базы данных PostgreSQL (размер требуемого хранилища зависит от периода, за который должны храниться данные).

Клиентские рабочие места не предъявляют жёстких требований: достаточно, чтобы на рабочем ПК работал любой из современных браузеров (Google Chrome, Mozilla Firefox, Microsoft Edge и др.) и производительности было достаточно для приемлемой задержки отклика с точки зрения пользователя.

2.1.2 Язык программирования и серверные фреймворки

Язык программирования Kotlin выбран для разработки серверных модулей, поскольку одна из активно развиваемых библиотек для работы с геохешированием S2 предназначена для работы в JVM (Java Virtual Machine – виртуальная машина Java). В качестве серверного фреймворка будет использован Spring Boot версии не ниже 3.5. Spring Boot, веб-фреймворк с открытым исходным кодом, является лидером среди фреймворков для построения серверных приложений для JVM, являясь надежным и удобным решением, развиваясь с участием сообщества разработчиков [22]. Также язык Kotlin обладает высокой производительностью несмотря на то, что работает в виртуальной машине, благодаря оптимизациям и JIT-компиляции.

2.1.3 Системы управления базами данных и кэширования

В качестве основной СУБД выбрана PostgreSQL 16 [13]. Определяющими факторами послужили:

- популярность СУБД, её активное развитие и богатая документация;
- высокий уровень производительности;
- наличие расширений для обработки географических данных (PostGIS), что может быть полезно при будущем развитии системы.
- Для кэширования и организации очередей сообщений выбраны два продукта:
 - Redis для кэширования и межсервисного взаимодействия. Его использование подтверждено успешным опытом в высоконагруженных системах [23], где требуется быстрое обновление данных без обращения к дисковой СУБД.

– Apache Kafka в качестве распределённого брокера сообщений, образующего ядро потоковой шины. Kafka обладает возможностью горизонтального масштабирования и репликации, что позволяет организовать надежную и производительную потоковую обработку данных [7].

2.1.4 Выбор используемых библиотек

Наиболее подходящим является метод геохеширования на основе библиотеки S2 Geometry Library For Java, который обеспечивает:

– представление географических областей в виде наборов дискретных ячеек (интервалов ячеек) что позволяет заменить проверку геометрического вхождения на быстрый бинарный поиск по предварительно рассчитанному интервальному индексу, хранимому в оперативной памяти;

– логарифмическую сложность проверки, предсказуемое время выполнения даже для сложных многоугольных зон;

– возможность гибко регулировать точность представления, управляя потреблением памяти.

Таким образом, будет использована библиотека S2 Geometry Library For Java [15].

2.1.5 Платформа клиентского веб-интерфейса

Клиентская часть должна быть реализована с применением компонентно-ориентированного фронтенд-фреймворка Vue.js на языке TypeScript. Основанием для такого выбора является:

– высокая скорость разработки динамических интерфейсов благодаря реактивной модели данных;

– компонентный подход, реализуемый фронтенд-фреймворком Vue и позволяющий переиспользовать код и разметку в разных частях приложения;

– наличие экосистемы готовых компонентов для отображения карт, построения таблиц и графиков, что особенно ценно для диспетчерских систем;

– строгая типизация языка TypeScript, снижающая количество ошибок на этапе разработки.

2.1.6 Инфраструктурные решения

Развертывание всех серверных компонентов в Docker-контейнерах обеспечивает идентичность окружения на этапах разработки, тестирования и промышленной эксплуатации, упрощает развёртывание и горизонтальное масштабирование. Для управления растущим количеством контейнеров будет применен Docker Compose [21]. Организация CI/CD пайплайна с использованием GitHub Actions или GitLab CI позволит автоматизировать сборку образов и их публикацию в реестр.

Таким образом, предложенный стек удовлетворяет функциональным и нефункциональным требованиям, сформулированным в разделе 1.2, и создаёт необходимый фундамент для реализации высокопроизводительной, горизонтально масштабируемой системы мониторинга.

2.2 Выбор средств разработки

2.2.1 Система контроля версий

В качестве системы контроля версий применяется Git [24]. Для хранения исходного кода и организации командной работы выбрана платформа GitHub. Сервис GitHub имеет функционал GitHub Actions, который позволяет автоматизировать тестирование, сборку и развертывание на основе данных репозитория.

2.2.2 Интегрированные среды разработки

Для серверной части системы, реализуемой на языке Kotlin, основной средой разработки служит OpenIde, являющаяся российской средой разработки, удовлетворяющей российскому законодательству и основанной на IDE IntelliJ IDEA компании JetBrains, которая также является автором языка Kotlin, благодаря чему обеспечивается широкая поддержка языка программирования, применяемых фреймворков и разнообразных расширений.

Для разработки клиентского веб-интерфейса на Vue.js и TypeScript применяется редактор Visual Studio Code. Его популярность в сообществе фронтенд-разработчиков и богатый выбор расширений (в том числе и от разработчиков Vue) делают его предпочтительным инструментом для быстрой и качественной работы с реактивными интерфейсами.

2.2.3 Система сборки и управления зависимостями

В серверных микросервисах используется Gradle с Kotlin DSL. Для выполнения сборки и публикации docker-образа использован плагин Gradle Google Jib. Для фронтенд-части в качестве сборщика применяется Vite [25]. Обе системы сборки являются наиболее распространенными среди новых проектов для своего стека технологий.

2.2.4 Инструменты тестирования

Обеспечение высокого качества разрабатываемой системы требует многоуровневого тестирования. Модульное тестирование серверных компонентов выполняется с использованием JUnit [26]. Для проверки REST API применяется Bruno, что даёт возможность создавать и выполнять коллекции запросов, проверяя корректность ответов.

3 Реализация и полученные результаты

3.1 Описание разработки программного обеспечения

3.1.1 Проектирование архитектуры системы

Проектирование архитектуры – процесс планирования внутреннего устройства системы: определение количества и типов подсистем, связей между ними. Для наглядного представления данного этапа проектирования была выбрана нотация диаграммы компонентов, которая представлена на рисунке 3.1.

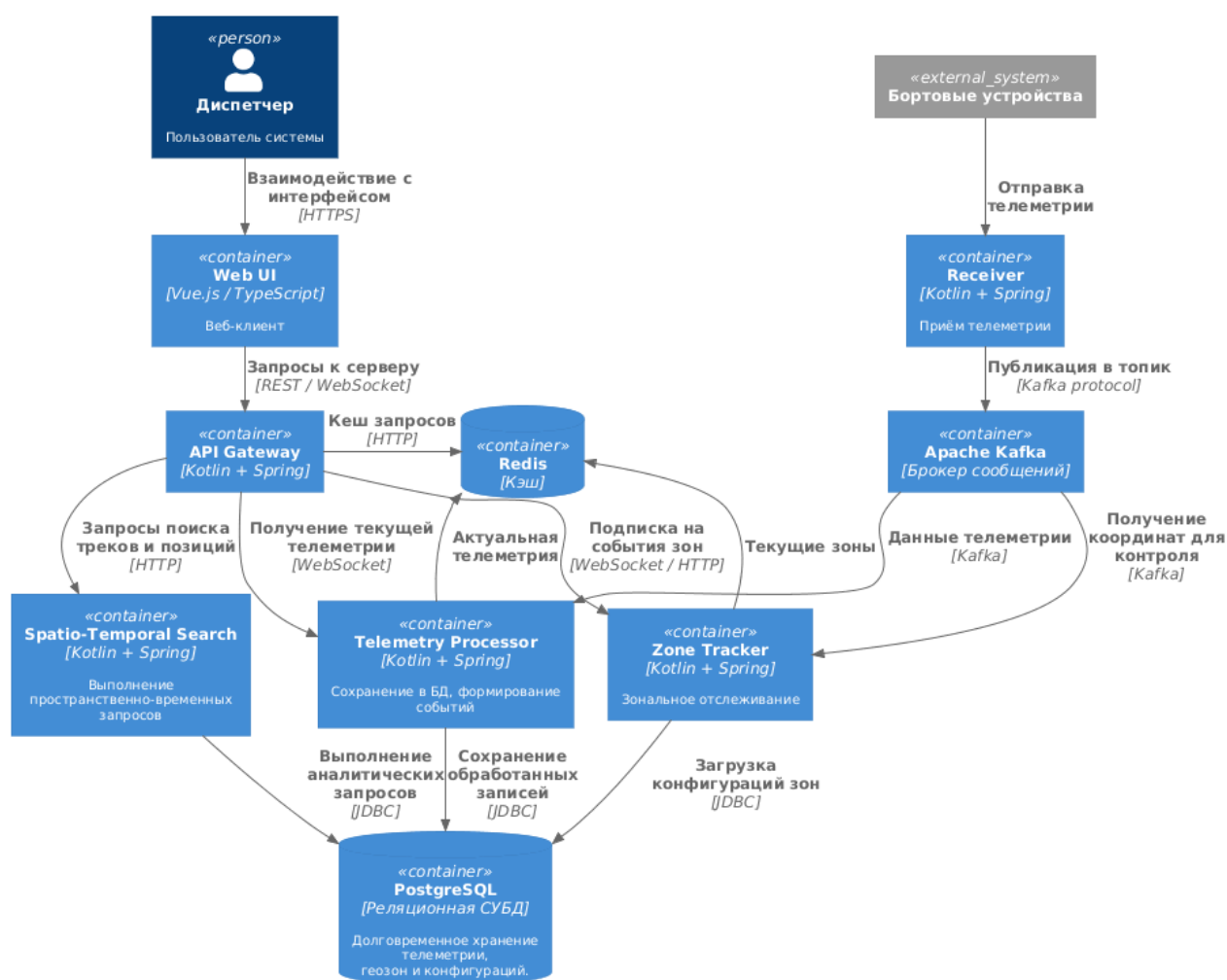


Рисунок 3.1 — Архитектура системы

Архитектура системы на рисунке 1 представлена в минимальном исполнении, когда каждый из модулей представлен в единственном экземпляре. При масштабировании системы количество модулей может быть увеличено:

могут быть добавлены дополнительные приемники, передатчики и обработчики телеметрии, генераторы событий.

3.1.2 Описание модулей системы

Далее будет приведено описание каждой из подсистем разрабатываемой системы, указано их назначение, значимые подробности об их внутреннем устройстве, связи с другими модулями, протоколы обмена, состав данных.

1. Клиентское веб-приложение (Web UI)

Клиентское веб-приложение, предоставляющее пользовательский интерфейс для мониторинга транспортных средств, просмотра треков, событий, ведения справочника геозон и других настроек.

Модуль осуществляет взаимодействия, представленные в таблице 3.1.

Таблица 3.1 — Связи клиентского модуля

Целевой модуль	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
Шлюз API	Отправка запросов и получение ответов в ходе любых операций пользователя	HTTP, WebSocket	HTTP: JSON с параметрами запросов, ответы в JSON (списки ТС, треки, события). WebSocket: сообщения с уведомлениями о событиях.

2. Шлюз API (API Gateway)

Единая точка входа для клиентского приложения. Маршрутизирует запросы к соответствующим модулям.

Связи модуля представлены в таблице 3.2.

Таблица 3.2 — Связи шлюза API

Целевой модуль	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
Обработчик телеметрии	Получение актуальной телеметрии и событий	WebSocket	Запрос: параметры фильтрации (список ТС) Ответ: поток актуальных телеметрических данных и событий
Модуль зонального отслеживания	Получение событий зон	WebSocket	Запрос: параметры фильтрации (список ТС) Ответ: поток событий геозон
Модуль пространственно-временного поиска	Поиск треков, ТС в заданном регионе за период, поиск телеметрии по ТС	HTTP	Запрос: фильтры по ТС, зона поиска. Ответ: список ТС. Запрос: ТС, временной интервал Ответ: массив телеметрических данных
Redis	Запрос закешированных данных	Redis	Картинки, конфигурация, метаданные объектов системы

3. Приемная служба (Receiver)

Приём телеметрических пакетов от бортовых устройств. Обеспечивает разбор входящих данных, отправку данных в брокер сообщений.

Внешние взаимодействия модуля приема телеметрических данных отображены в таблице 3.3.

Таблица 3.3 — Связи приемной службы

Целевой контейнер	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
Брокер сообщений	Передача принятых телеметрических данных	Kafka protocol	Разобранные телеметрические данные

4. Брокер сообщений (Apache Kafka)

Центральная асинхронная шина сообщений, развязывающая источники и потребителей данных. Обеспечивает гарантированную доставку, масштабируемость и отказоустойчивость.

Основным понятием обмена, используемом в Kafka, является топик (topic). В топике хранятся сообщения - бинарные данные ключ-значение. Каждый из топиков может быть распределен между несколькими серверами кластера. Сообщения в топике хранятся определенное ограниченное время, после которого удаляются (также возможно ограничение по размеру топика), до удаления возможна перечитка данных. Топик делится на партиции, распределение по которым происходит с помощью хеширования ключа сообщения. Kafka поддерживает практически неограниченное расширение кластера – до сотен серверов в одном кластере, что делает способность к горизонтальному расширению очень высокой.

Модули, работающие с брокером сообщений, могут выступать в качестве производителей либо потребителей сообщений. При доставке сообщений в большинстве случаев используется модель доставки по подписке, когда между потребителем и брокером постоянно сохраняется соединение, и в момент поступления события брокер инициирует отправку сообщения потребителю. Такая модель позволяет серверу не знать адресов потребителей, но при этом с минимальной задержкой обеспечивать доставку сообщений по назначению.

5. Обработчик телеметрии (Telemetry Processor)

Потребляет поток телеметрических данных из брокера сообщений, сохраняет обработанные записи в базу данных и в Redis. Формирует для каждого телеметрического сообщения S2-ключ заданного уровня для использования в пространственно-временном поиске.

Обмен данными для обработчика телеметрии описан в таблице 3.4.

Таблица 3.4 — Связи обработчика телеметрии

Целевой модуль	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
База данных	Сохранение нормализованных телеметрических записей для последующего анализа и поиска.	JDBC	Основные данные (данные записи); дополнительные данные (данные подзаписей)
Redis	Сохранение актуального местоположения ТС	Redis	Ключ – ТС, значение – наиболее актуальные телеметрические данные (географические данные и данные датчиков)

6. Модуль зонального отслеживания

Реализует зональное отслеживание в реальном времени. Потребляет телеметрические данные из общего потока Kafka, с помощью пространственного индекса на базе Google S2 определяет факт вхождения/выхода ТС в геозоны и генерирует соответствующие события. Описание алгоритма геохеширования зон, формирования индекса в памяти и использования его при работе приведено в п. 3.1.3.

Внешние связи модуля показаны в таблице 3.5.

Таблица 3.5 — Связи модуля зонального отслеживания

Целевой контейнер	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
PostgreSQL	Загрузка актуального списка геозон и их параметров Сохранение событий посещений зон	JDBC	Данные геозон: тип, объекты описания (точки) Сохранение записи о входе/выходе из геозоны
Redis	Сохранение/загрузка зон, в которых ТС находится в данный момент	Redis	Ключ – ТС, значение – список текущих зон

7. Модуль пространственно-временного поиска

Модуль выполняет поиск телеметрических данных по географическому интервалу и временному интервалу; по ТС и временному интервалу.

Алгоритм работы пространственно-временного поиска состоит из этапов:

1. определение перечня зон S2 определенного минимального уровня, которые полностью покрывают произвольную зону поиска, задаваемую полигоном (набором точек);
2. представление всех найденных зон S2 в виде интервалов (двух ячеек минимального уровня, описывающего более крупный уровень);
3. слияние интервалов (представление двух и более смежных интервалов одним);
4. выполнение запроса к таблице телеметрии с условием по временному промежутку и условием вхождения S2 ключа телеметрии в один из интервалов, рассчитанных на предыдущих этапах.

В результате выполнения описанных шагов определяется перечень данных: транспортное средство, начальное время нахождения ТС в искомом регионе, конечное время нахождения ТС в искомом регионе. Стоит отметить, что

точность определения напрямую зависит от уровня используемого минимального уровня S2-зоны.

Связи модуля пространственно-временного поиска описаны в таблице 3.6.

Таблица 3.6 — Связи модуля пространственно-временного поиска

Целевой контейнер	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
PostgreSQL	Выполнение пространственно-временных SQL-запросов с использованием составного индекса Поиск телеметрических данных для ТС	JDBC	Запрос: временной интервал, пространственный интервал Ответ: ТС, находившиеся в заданном временном промежутке в заданный пространственный интервал Запрос: ТС, временной интервал Ответ: телеметрические данные

8. Redis

Высокопроизводительное хранилище в оперативной памяти для кэширования оперативной информации, обеспечивающее снижение издержек на работу базы данных при работе системы. Используется шлюзом API для уменьшения количества запросов к другим сервисам, и как следствие, для повышения производительности системы, а также обработчиком телеметрических данных для хранения актуальной информации о местоположении ТС.

9. PostgreSQL

Основное реляционное хранилище для долговременного хранения нормализованных телеметрических данных, конфигураций геозон, ведения справочников, настроек событий, событий и прочих данных. Выбрана версия PostgreSQL 18 – последняя стабильная версия на момент проектирования системы.

Обслуживает SQL-запросы от других модулей. Состав хранимых данных будет описан при описании структур данных.

Встроенные возможности горизонтального масштабирования PostgreSQL ограничены, при необходимости применяются дополнительные решения для шардирования баз данных.

3.1.3 Пространственный индекс

В ходе проектирования программного продукта был разработан индекс пространственного поиска. Индекс работает с помощью системы геохеширования S2. Работу с индексом можно разделить на два этапа:

1. построение индекса для списка регионов S2;
2. использование индекса для поиска регионов, содержащих заданную точку.

Регион S2 – область на поверхности земли, которая может быть заполнена ячейками S2. Для покрытия региона в библиотеке S2 присутствует класс S2RegionCoverer. Процесс покрытия настраивается параметрами:

1. максимальный уровень зон заполнения;
2. минимальный уровень зон заполнения;
3. максимальное количество зон заполнения;
4. тип заполнения – избыточный/внутренний.

Алгоритм заполнения с учетом параметров заполняет заданный регион зонами так, чтобы покрыть максимальное количество площади региона и при этом задействовать минимально возможное количество пространства, не относящегося к региону. В зависимости от типа заполнения зоны либо

размещаются только внутри региона, оставляя минимально возможное незаполненное место, либо, если выбрано избыточное кодирование, могут быть размещены снаружи региона, занимая минимально возможное избыточное пространство.

На рисунке 3.2 представлен пример заполнения региона сложной геометрии при различных параметрах алгоритма избыточного заполнения. Максимальное количество зон последовательно возрастает от первого примера к третьему, что позволяет заполнить регион с гораздо более высокой точностью.

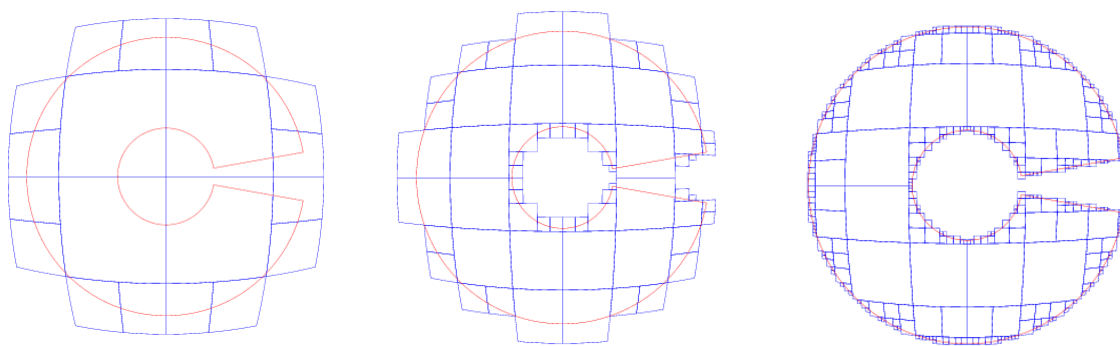


Рисунок 3.2 — Заполнение региона при различных параметрах

Каждая из S2-зон обладает следующим свойством: её можно описать как интервал $[c; \text{по})$ последовательных значений-ключей зон более мелкого (дочернего уровня), причем сколь угодно мелкого. Если какая-либо зона A определенного уровня попадает в интервал BC $[c; \text{по})$ того же уровня, то благодаря свойствам S2-индексации можно с уверенностью сказать, что зона A входит внутрь S2-зоны, описываемой интервалом BC.

Более того, соседние смежные интервалы AB и BC можно объединить в один большой интервал AC, что позволит сократить количество интервалов и ускорить поиск.

Из вышеперечисленных фактов можно заметить, что задача определения вхождения точки в один или несколько регионов может быть сведена к определению вхождения S2-ключа этой точки в один из интервалов,

описывающих S2-зоны, заполняющие один или несколько регионов, после чего с некоторой точностью (соответствующей выбранному уровню S2-ключей интервалов) можно сделать вывод о вхождении этой точки в соответствующие регионы. Если из регионов составить отсортированный список непересекающихся интервалов, то поиск в нем по значению будет иметь временную сложность $O(\log N)$, где N – количество интервалов в отсортированном списке.

Для дополнительной точности на последнем этапе поиска можно выполнить геометрическую проверку на вхождение точки в каждый из найденных регионов, чтобы исключить ложноположительные срабатывания вхождения на границах регионов. Это наиболее актуально в тех случаях, если для составления индекса выбирается достаточно крупный уровень S2.

На рисунках 3.3, 3.4, 3.5 показана блок-схема инициализации и использования индекса.

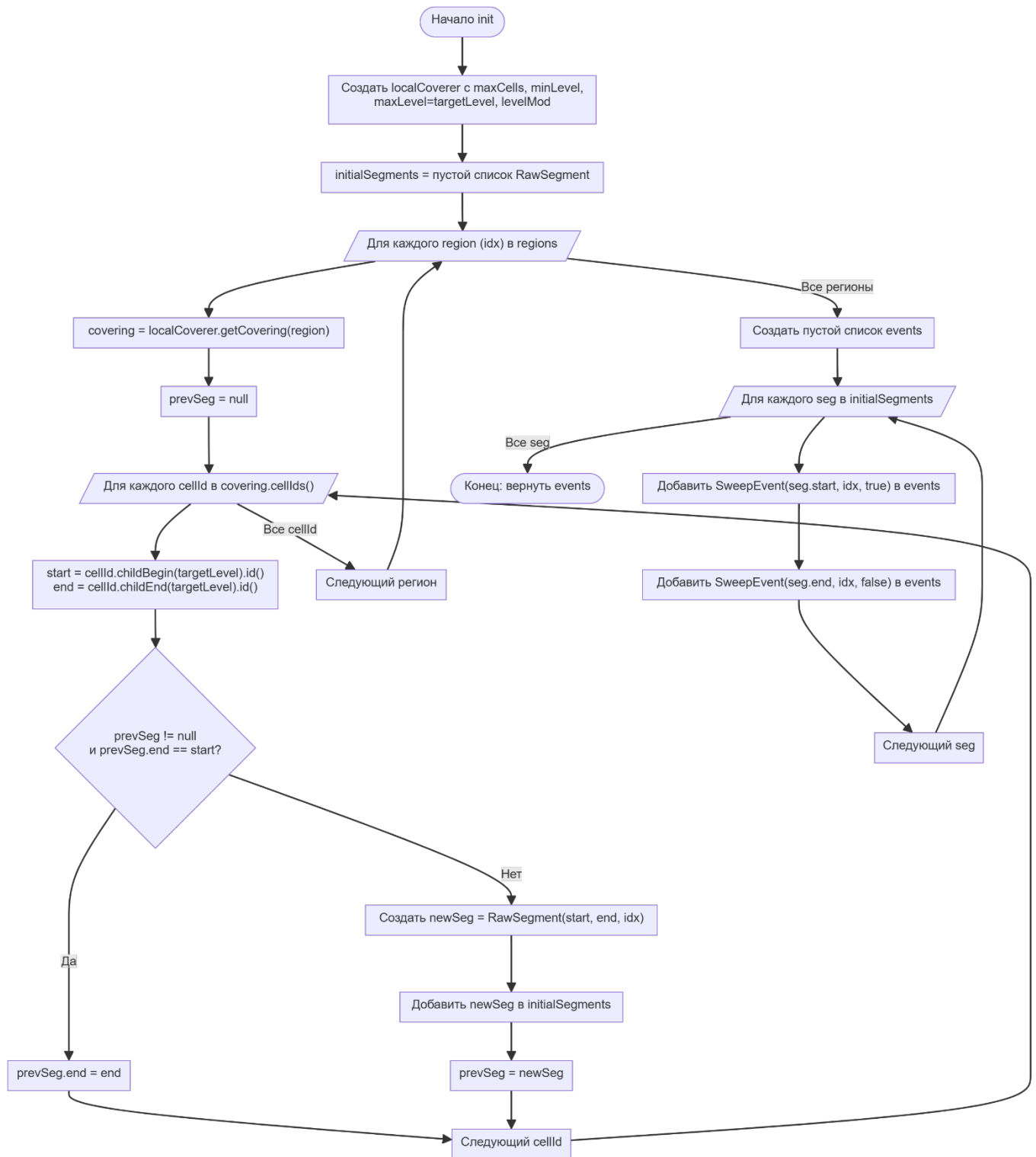


Рисунок 3.3 — Блок-схема инициализации индекса (ч.1)

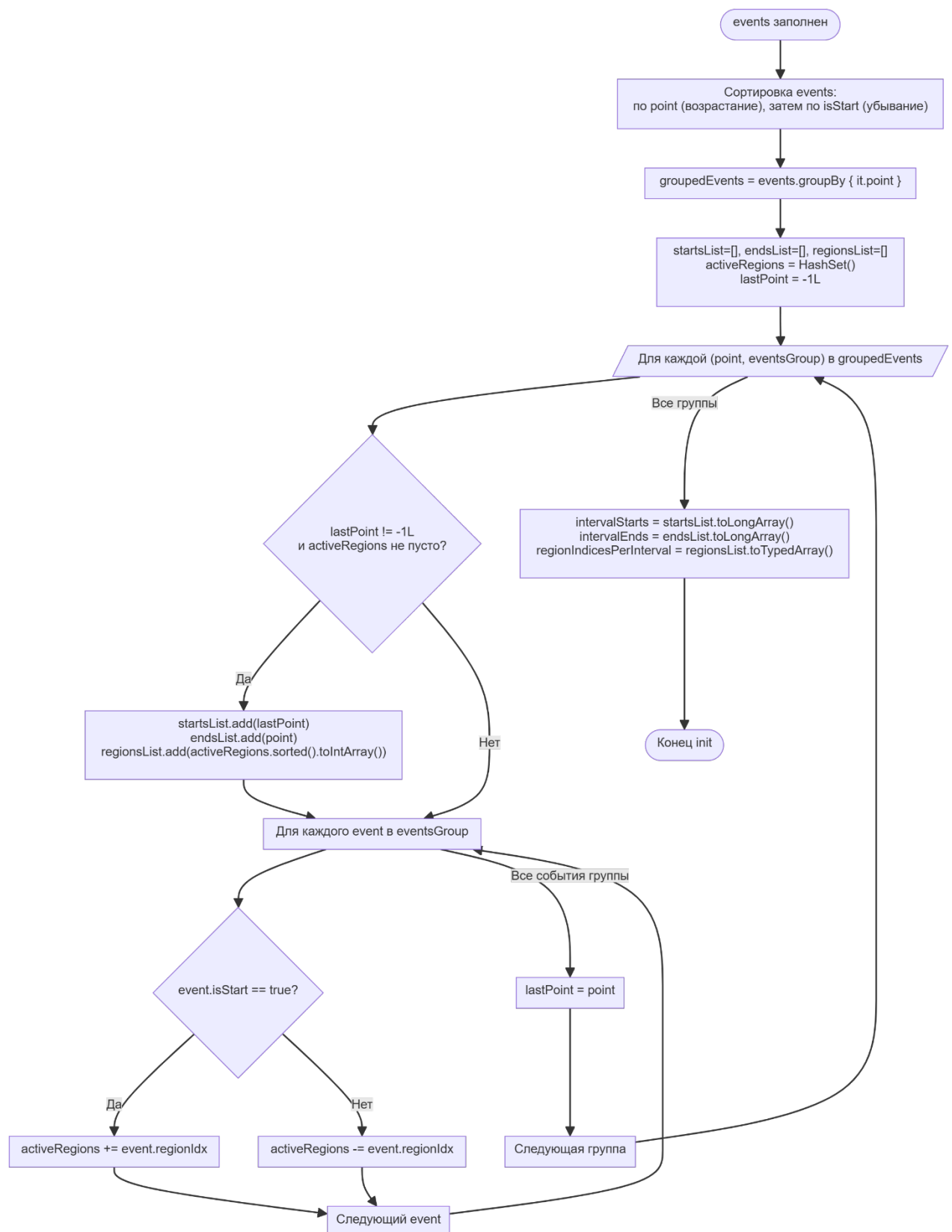


Рисунок 3.4 — Блок-схема инициализации индекса (ч.2)

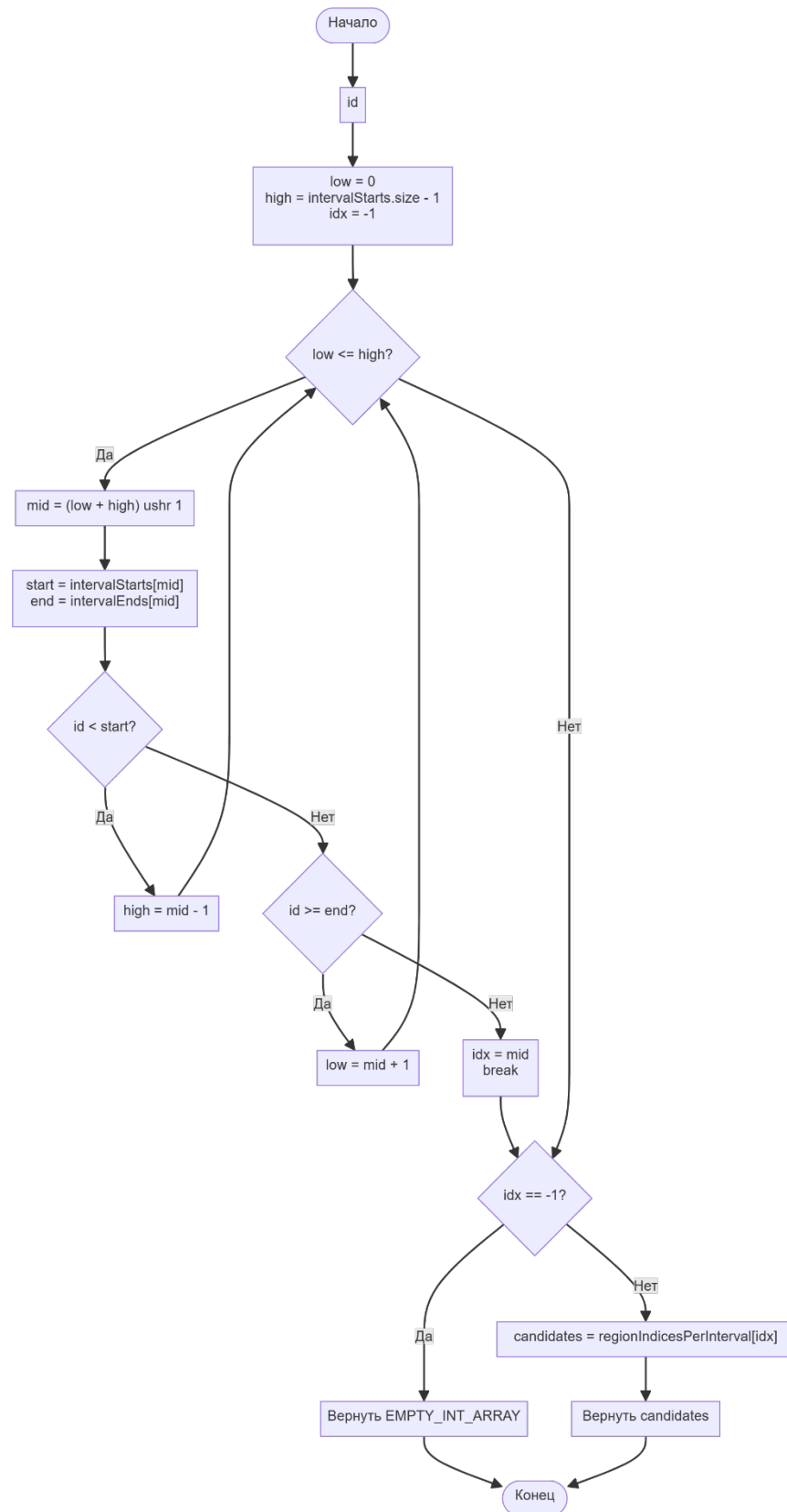


Рисунок 3.5 — Блок-схема выполнения поиска регионов в индексе

В результате построения индекса формируется три списка: отсортированный список с началами интервалов, список с концами интервалов, список со списками индексов регионов, в которых содержится соответствующий интервал. Каждый из списков представляет из себя массив со сложностью произвольного доступа по индексу за $O(1)$ по времени. Логически структуру данных можно отобразить в виде схемы (рис. 3.6).

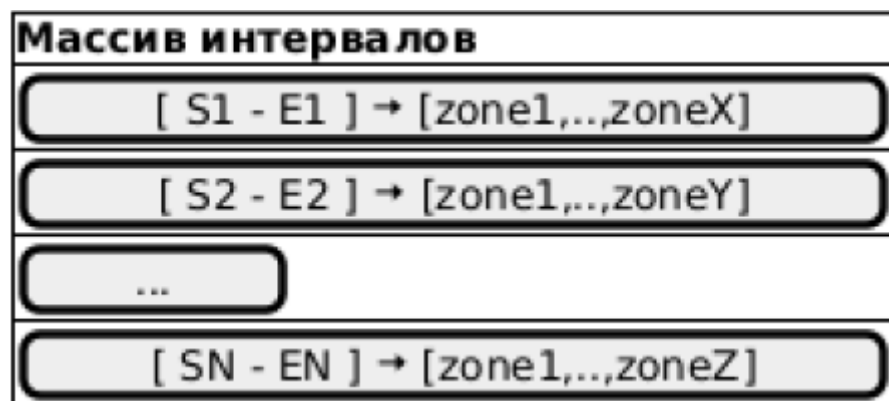


Рисунок 3.6 — Логическая структура пространственного индекса

При поиске для точки в пространстве рассчитывается S2-номер (id) соответствующего уровня, после чего производится бинарный поиск в массиве. Если найден интервал – соответствующий список регионов берется как стартовый для этапа геометрической проверки вхождения.

В составе библиотеки S2 для Java присутствует пространственный индекс S2ShapeIndex, который может решить аналогичную задачу с помощью запроса S2ContainsPointQuery. В ходе тестирования производительности индексов было выявлено, что разработанный в ходе данной работы индекс S2ZoneQuery превосходит встроенный индекс по производительности примерно на 30% при параметрах покрытия регионов:

- минимальный уровень = 1;
- максимальный уровень = 24;
- максимальное количество зон для покрытия региона – 200.

Исходный код индекса приведен в составе Приложения Б.

3.2 Описание структур данных

Хранение данных в системе обеспечивается посредством двух модулей: базы данных и брокером сообщений. Поскольку данные, передаваемые в брокере сообщений, также представлены и в базе данных, достаточно будет описания структур данных в БД для полного описания структур всех сущностей, существующих в системе.

3.2.1 Диаграмма отношений сущностей

Для описания БД на рисунке 3.7 представлена диаграмма отношений сущностей (ER-diagram) – логическая схема базы данных.

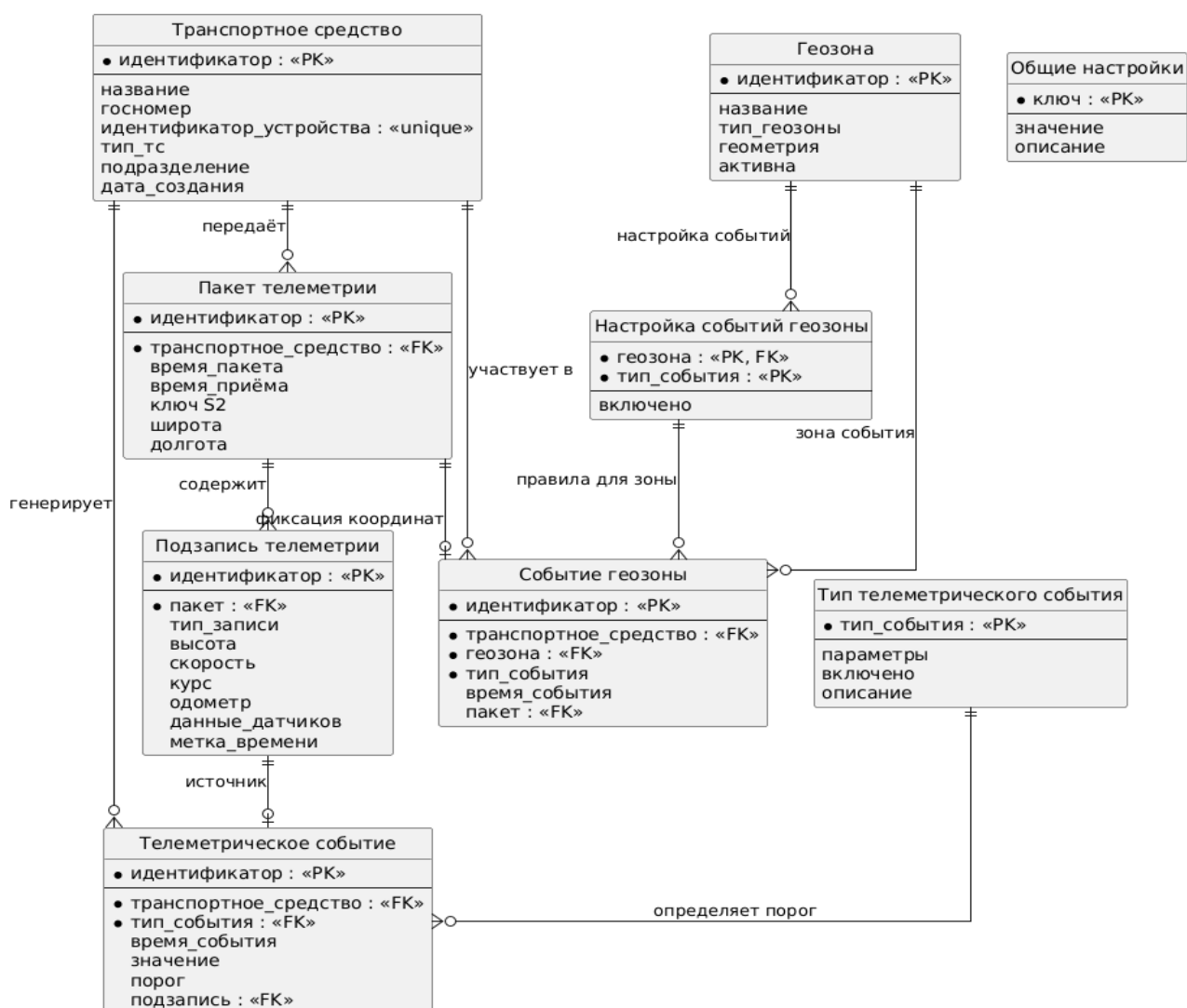


Рисунок 3.7 — Логическая схема базы данных

На рисунке 3.8 представлена физическая модель базы данных с указанием конкретных имен колонок и типов данных для СУБД PostgreSQL.

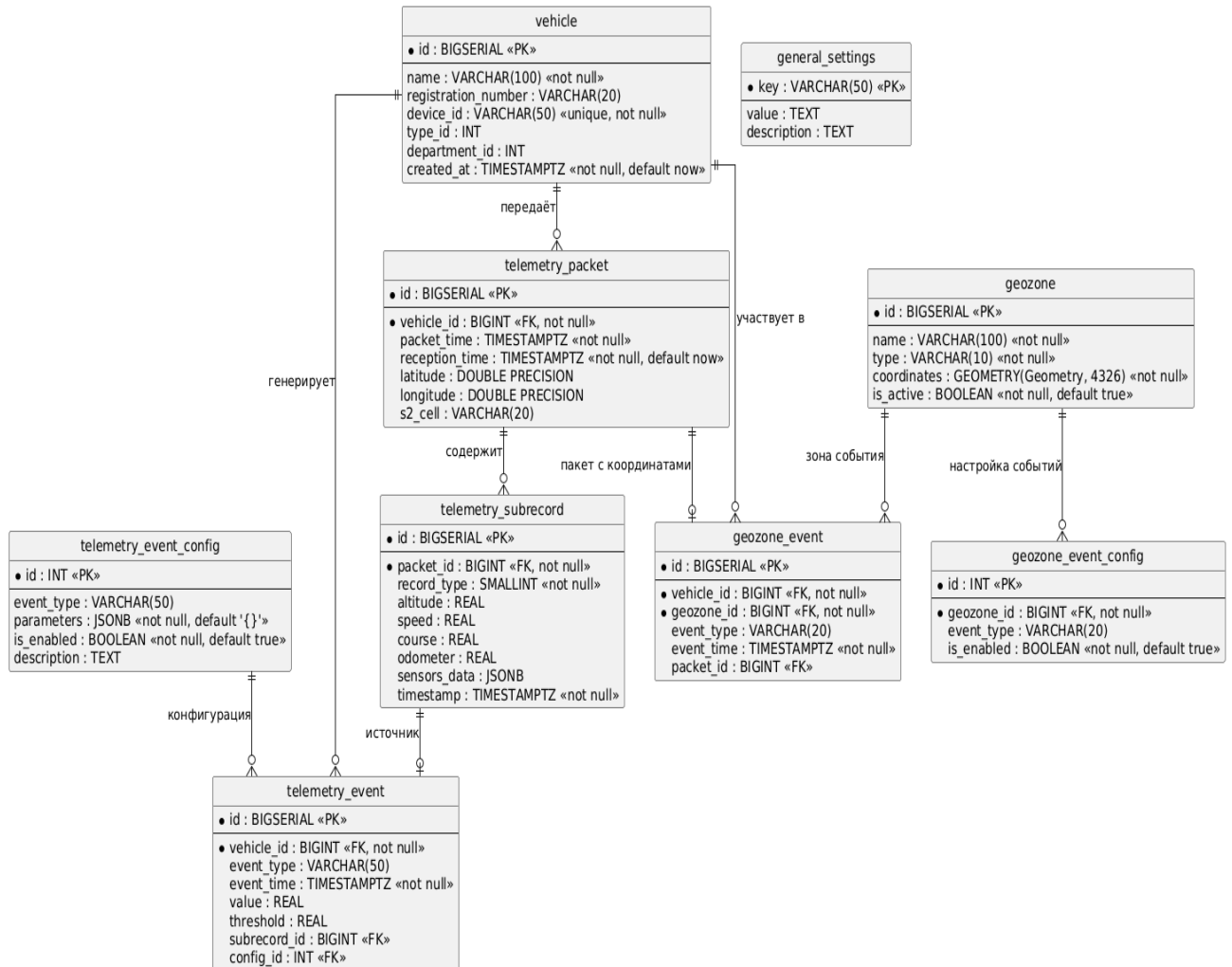


Рисунок 3.8 — Физическая схема базы данных

3.2.2 Словарь данных

Детальное описание полей реляционной базы данных представлено в таблицах 3.7-3.15 для каждой из таблиц.

1. Справочник транспортных средств (vehicles)

Таблица 3.7 — Справочник транспортных средств

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, значение автоинкрементируется, начиная с 1
name	varchar(100)	Наименование транспортного средства (отображаемое имя)
registration_number	varchar(20)	Государственный регистрационный знак
device_id	varchar(50) UNIQUE	Идентификатор бортового устройства, уникален для каждого ТС
type_id	int	Идентификатор типа транспортного средства (справочник)
department_id	int (FK)	Внешний ключ к таблице подразделений (departments)
created_at	timestamp	Дата и время создания записи

2. Справочник геозон (geozones)

Таблица 3.8 — Справочник геозон

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, автоинкремент
name	varchar(100)	Название геозоны
type	varchar(10)	'polygon' — многоугольник, 'circle' — окружность
coordinates	geometry	Координаты: массив вершин, для circle — центр и радиус
is_active	boolean	Признак активности геозоны

3. Данные телеметрии (telemetry_packets)

Таблица 3.9 — Данные телеметрии

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, автоинкремент
vehicle_id	bigint (FK)	Внешний ключ к таблице vehicles
packet_time	timestamp	Время из пакета, установленное бортовым устройством
reception_time	timestamp	Время приёма пакета сервером
latitude	double precision	Широта из пакета (позиционная подзапись или усреднённая)
longitude	double precision	Долгота из пакета
s2_cell	varchar(20)	Ключ S2 (идентификатор ячейки), вычисленный по координатам с уровнем, заданным в general_settings. Используется для пространственного поиска.

4. Дополнительные данные телеметрии (telemetry_subrecords)

Таблица 3.10 — Дополнительные данные телеметрии

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, автоинкремент
packet_id	bigint (FK)	Внешний ключ к таблице telemetry_packets

Продолжение таблицы 3.10

Наименование	Тип данных	Значение
record_type	smallint	Тип подзаписи (0 — позиционная, 1 — состояние и т.д.)
altitude	real	Высота над уровнем моря (м)
speed	real	Скорость (км/ч)
course	real	Направление движения (градусы)
odometer	real	Показания одометра (км)
sensors_data	jsonb	Данные дополнительных датчиков в структурированном виде
timestamp	timestamp	Время подзаписи (если отличается от времени пакета)

5. События телеметрии (telemetry_events)

Таблица 3.11 — События телеметрии

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, автоинкремент
vehicle_id	bigint (FK)	Внешний ключ к таблице vehicles
event_type	varchar(50)	Тип события (например, 'speed_exceed', 'harsh_braking')
event_time	timestamp	Момент возникновения события
value	real	Фактическое значение параметра, вызвавшего событие
threshold	real	Пороговое значение, заданное в настройках

Продолжение таблицы 3.11

Наименование	Тип данных	Значение
subrecord_id	bigint (FK) опционально	Ссылка на конкретную подзапись, в которой зафиксировано событие
config_id	int (FK)	Внешний ключ к таблице telemetry_event_config

6. События отслеживания геозон (geozone_events)

Таблица 3.12 — События геозон

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, автоинкремент
vehicle_id	bigint (FK)	Внешний ключ к таблице vehicles
geozone_id	bigint (FK)	Внешний ключ к таблице geozones
event_type	varchar(20)	Тип события: 'enter' — вход в зону, 'leave' — выход из зоны
event_time	timestamp	Время возникновения события
subrecord_id	bigint (FK) опционально	Ссылка на подзапись с координатами, вызвавшую событие

7. Настройки событий телеметрии (telemetry_event_config)

Таблица 3.13 — Настройки событий телеметрии

Наименование	Тип данных	Значение
id	int	Первичный ключ, автоинкремент
event_type	varchar(50)	Тип телеметрического события
parameters	jsonb	Параметры события (пороги, условия, длительность и т.д.)
is_enabled	boolean	Признак активности данного типа событий
description	text	Текстовое описание конфигурации

8. Настройки событий геозон (geozone_event_config)

Таблица 3.14 — Настройки событий геозон

Наименование	Тип данных	Значение
id	int	Первичный ключ, автоинкремент
geozone_id	bigint (FK)	Внешний ключ к таблице geozones
event_type	varchar(20)	Тип события, в зависимости от которого меняется логика фиксации события
is_enabled	boolean	Признак активности конфигурации

9. Общие настройки (general_settings)

Таблица 3.15 — Общие настройки

Наименование	Тип данных	Значение
key	varchar(50) (PK)	Первичный ключ – строковый идентификатор настройки (например, 'geohash_level_db')
value	text	Значение настройки
description	text	Описание назначения параметра

3.3 Описание разработки интерфейса пользователя

Интерфейс пользователя является графическим пользовательским интерфейсом, выполненным в веб-среде. Пользователь выполняет взаимодействие с приложением с помощью веб-браузера. Клиентское приложение является одностраничным приложением – загруженная страница обладает высоким уровнем интерактивности, выполняя запросы данных с помощью сценариев страницы, без загрузок новых страниц и перезагрузок текущей страницы.

Разрабатываемая система получила название «Линкей» (лат. Lynceus), в честь одного из персонажей древнегреческой мифологии – это название отображается в интерфейсе.

В ходе использования системы отслеживания пользователи будут выполнять несколько групп задач.

3.3.1 Интерфейс ведения справочника объектов

Для определения перечня объектов мониторинга необходимо предоставить возможность создания, редактирования, обновления данных об этих объектах. В пользовательском интерфейсе этот функционал доступен в меню «Устройства», в котором доступен просмотр существующих объектов мониторинга (рисунок 3.9) и их редактирование (рисунок 3.10).

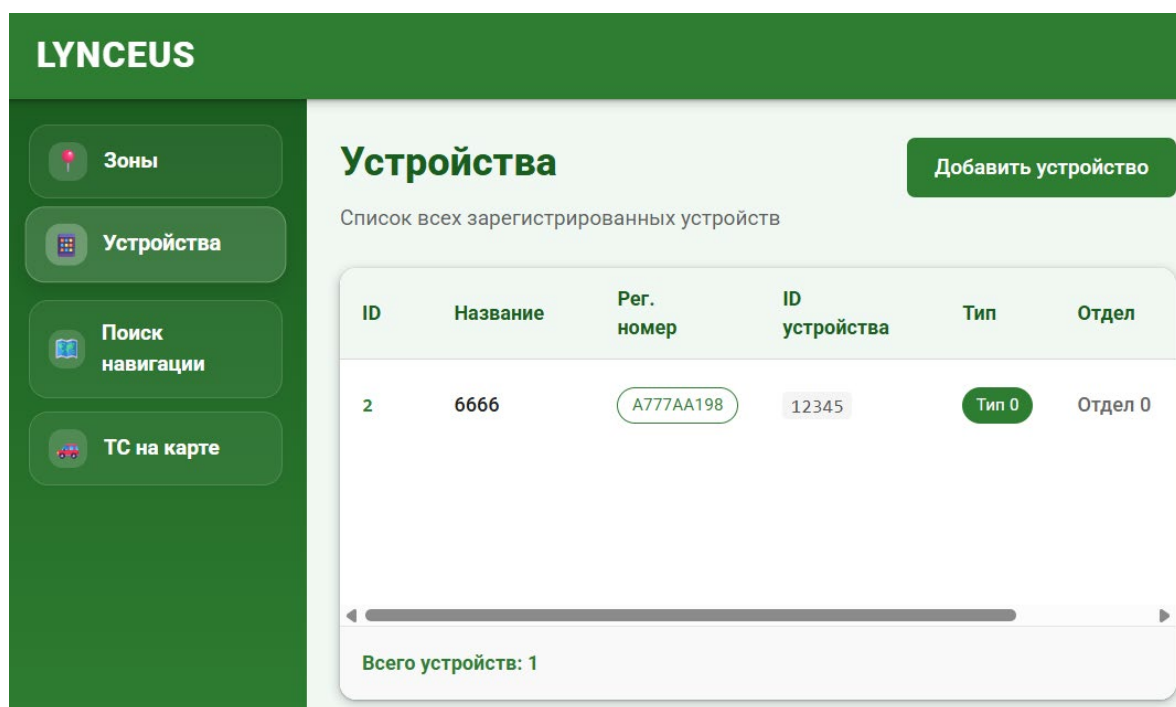


Рисунок 3.9 — Интерфейс просмотра справочника

The screenshot shows a modal form titled 'Добавить новое устройство' (Add new device). It contains several input fields with red asterisks indicating required fields:

- Название устройства ** (Device name): Input field with placeholder 'Введите название'.
- Регистрационный номер ** (Registration number): Input field with placeholder 'Введите регистрационный номер'.
- ID устройства (уникальный) ** (Device ID (unique)): Input field with placeholder 'Введите уникальный ID устройства'.
- Тип устройства (ID) ** (Device type (ID)): Input field with value '0'.
- Отдел (ID) ** (Department (ID)): Input field with value '0'.

At the bottom of the form are two buttons: 'Отмена' (Cancel) and 'Создать устройство' (Create device).

Рисунок 3.10 — Интерфейс добавления и редактирования объекта мониторинга

Для настройки географических зон отслеживания пользователь должен воспользоваться меню «Зоны», где доступен просмотр зон на карте с метаданными (рис. 3.11) и их редактирование (рис. 3.12).

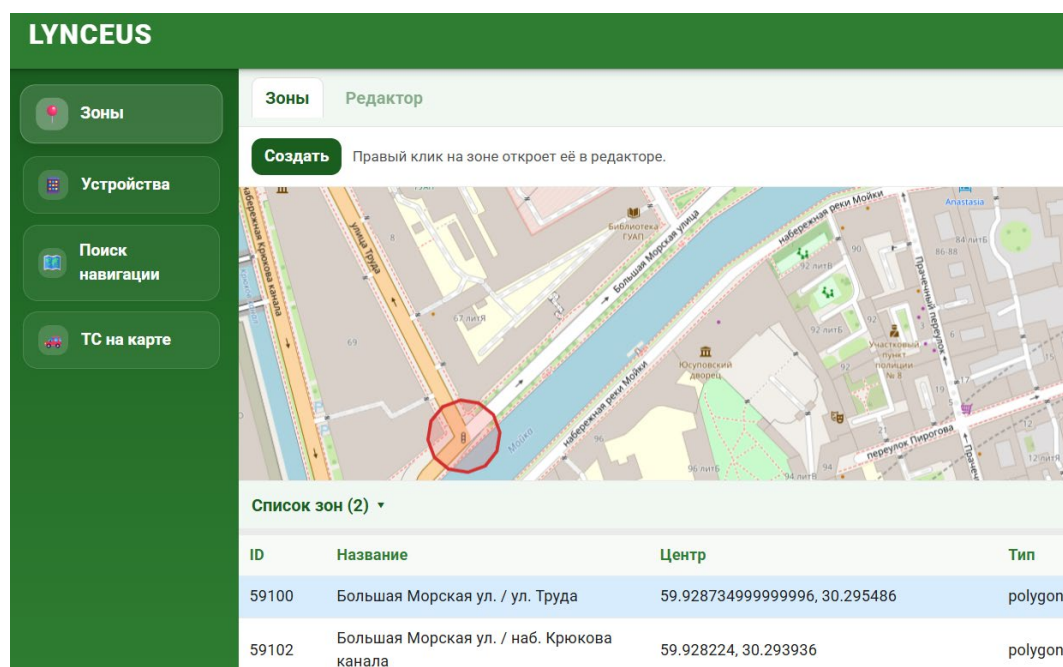


Рисунок 3.11 — Просмотр геозон на карте

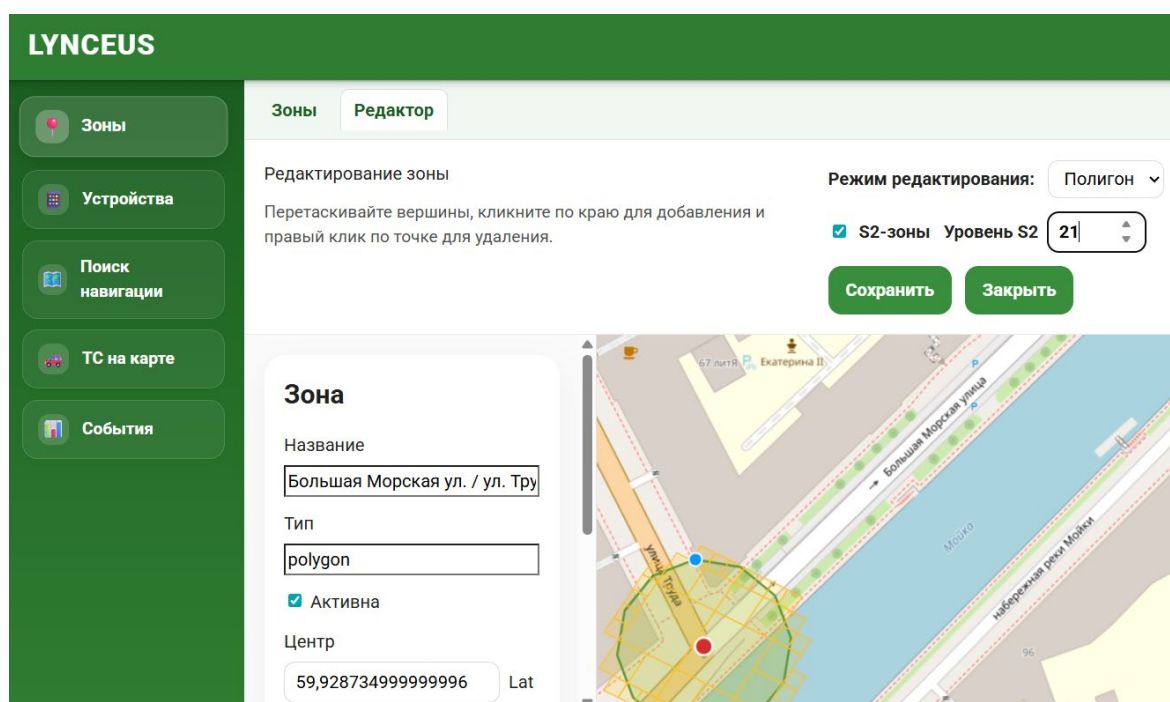


Рисунок 3.12 — Редактирование геозон

3.3.2 Интерфейс мониторинга транспортных средств

Для определения наиболее актуального местоположения ТС пользователю предоставлена возможность просмотра карты, на которой метками отображена позиция и направление движения транспортных средств (рисунок 3.13). Положение устройств на карте динамически обновляется в видимом пользователю пространстве карты по мере поступления новых телеметрических данных. При наведении указателя мыши на маркер отображается всплывающее окно с телеметрическими данными.

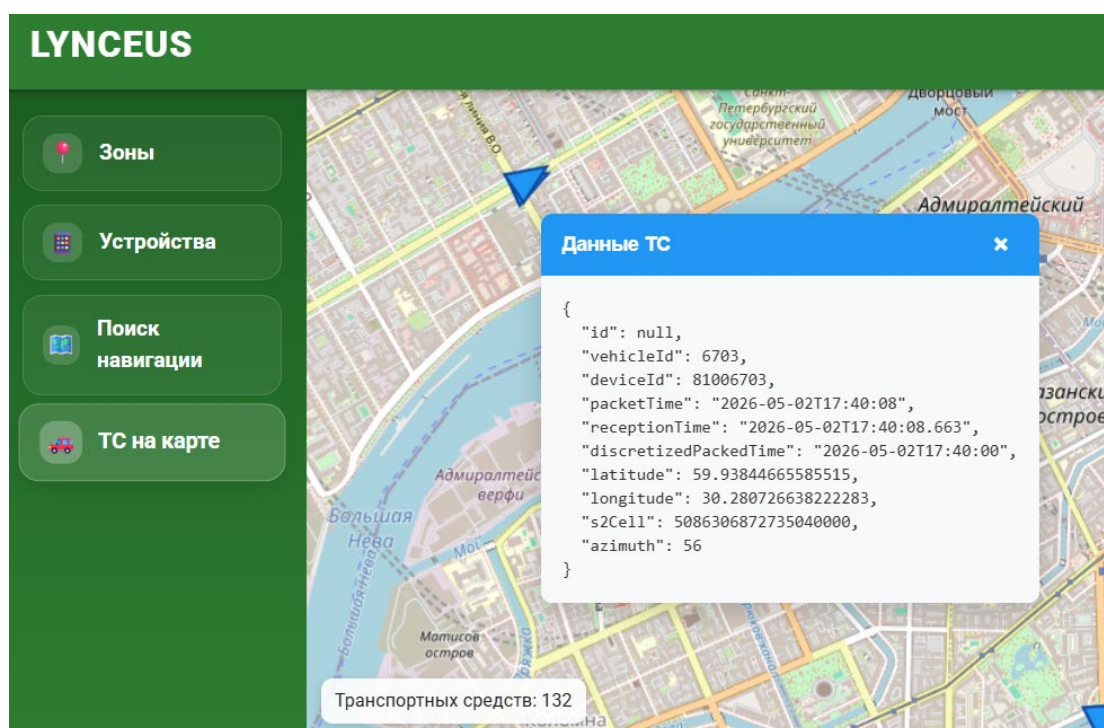


Рисунок 3.13 — Актуальное местоположение на карте

Для просмотра исторических телеметрических данных может быть использовано меню «Поиск навигации». На вкладке «Навигация устройства» может быть получена история местоположения за период времени для указанного устройства (рис. 3.14).

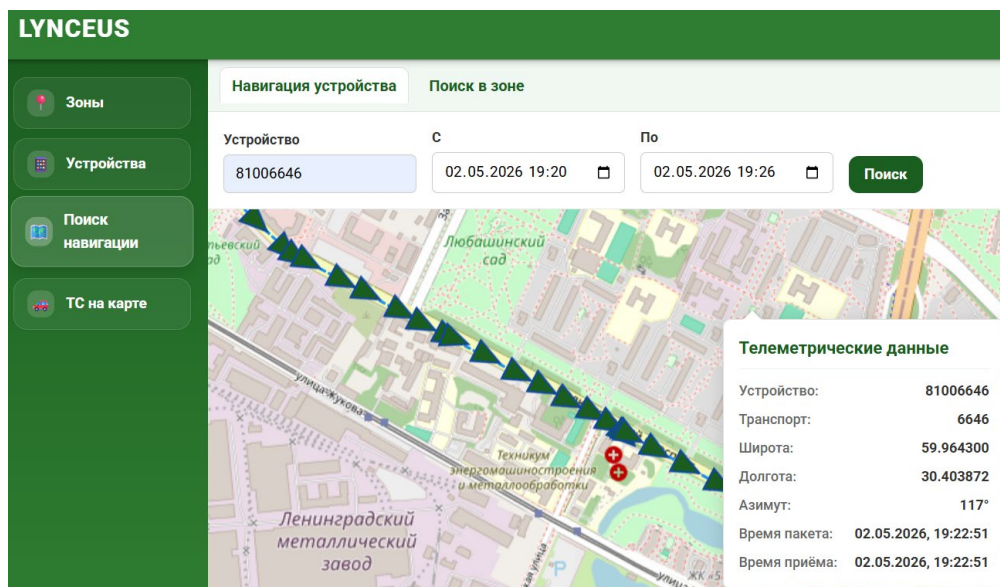


Рисунок 3.14 — Исторические телеметрические данные

Одна из центральных функций системы – пространственно-временной поиск исторических телеметрических данных. Эта функция доступна на вкладке «Поиск в зоне» (рисунок 3.15). Пользователем указывается временной период и зона поиска (четырёхугольник), после чего выполняется поиск.

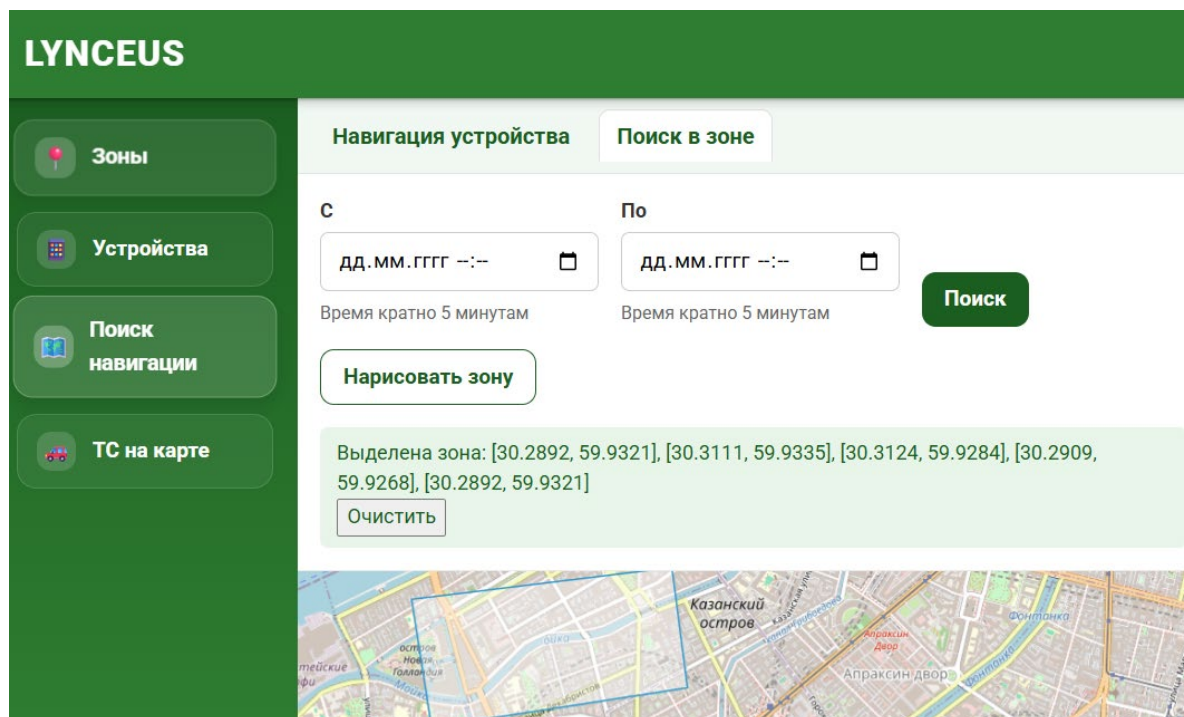


Рисунок 3.15 — Пространственно-временной поиск

После выполнения поиска отображается список устройств с указанием временного периода, в котором устройство было в зоне (рисунок 3.16). С помощью кнопки «Построить трек» доступен быстрый запрос исторических навигационных данных на вкладке «Навигация устройства».

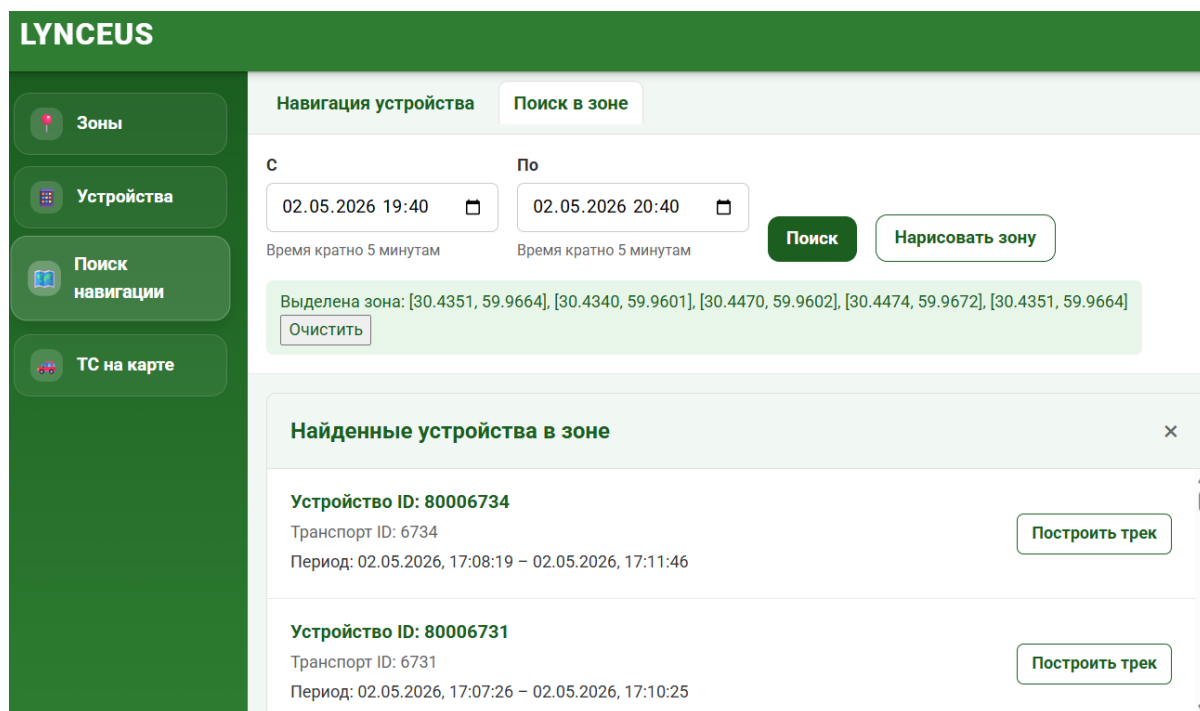


Рисунок 3.16 — Результат пространственно-временного поиска

На основе созданных в справочнике геозон формируются события входа в зону и выхода из зоны для устройств. Пользователю доступен просмотр формируемых событий в меню «События». События показываются по мере поступления новых данных. Внешний вид интерфейса показан на рисунке 3.17.

Вход/ Выход	ID устройства	ID ТС	ID зоны	Название зоны	Дата и время
Вход	81006577	ТС 6577	Зона 63859	3 КМ	03.05.2026, 12:22:51
Выход	81006667	ТС 6667	Зона 65489	МОРСКАЯ НАБ., 19	03.05.2026, 12:22:51
Вход	80006617	ТС 6617	Зона 51539	УЛ. БРАТЬЕВ РАДЧЕНКО, УГОЛ ПР. ЛЕНИ...	03.05.2026, 12:22:50
Выход	81006715	ТС 6715	Зона 64452	ЛЕНСКАЯ УЛ., 18	03.05.2026, 12:22:50
Вход	81006725	ТС 6725	Зона 62874	ГИПЕРМАРКЕТ "ЛЕНТА"	03.05.2026, 12:22:50

Рисунок 3.17 — Отображение событий

4 Тестирование и ввод в эксплуатацию

4.1 Тестирование программного обеспечения

Целью тестирования программного обеспечения является установление результатов разработки: устанавливается уровень качества программного обеспечения, проверяется соответствие требованиям, определенным на предыдущих этапах проектирования.

В современной разработке тестирование ПО принято разделять на различные виды по разным критериям, например: по уровням детализации (модульное, интеграционное, системное), по целям (функциональное, нефункциональное), по степени автоматизации (ручное, автоматизированное) и др.

В ходе выполнения работ по проекту для тестирования были выбраны следующие подходы:

- автоматизированное модульное тестирование с расчетом покрытия;
- ручное функциональное тестирование;
- автоматизированное тестирование производительности (нефункциональное тестирование).

4.1.1 Автоматизированное модульное тестирование

Автоматизированное модульное тестирование призвано обеспечить быстрое, надежное и повторяемое подтверждение корректности работы отдельных компонентов программы. Технически такое тестирование представляет из себя набор модулей программы, использующих специальные библиотеки для проведения тестирования и запускающихся автоматически при выполнении обновления исходных кодов в репозиториях систем контроля версий (например, GitLab, GitHub).

Большинство модулей разработанного продукта выполнено на языке Kotlin с использованием серверного фреймворка Spring Boot. Этот фреймворк имеет в своем составе компонент `spring-boot-starter-test`, который позволяет создавать тесты с помощью включенного в его состав решения Junit.

Запуск тестов производится с помощью задач (tasks) используемой системы сборки Gradle.

Пример исходного кода модульного теста приведен на рисунке 4.1.

```
@Test
fun `mergeAdjacentRanges should merge adjacent ranges with difference of 1`() {
    // Given
    val ranges = listOf(
        Pair(100L, 200L),
        Pair(201L, 300L) // Adjacent (200 + 1 = 201)
    )

    // Use reflection to test private method
    val method = TelemetryQueryService::class.java.getDeclaredMethod(
        name = "mergeAdjacentRanges",
        ...parameterTypes = List::class.java
    )
    method.isAccessible = true

    // When
    val result = method.invoke(
        obj = telemetryQueryService,
        | ...args = ranges) as List<Pair<Long, Long>>

    // Then
    assertEquals( expected = 1, actual = result.size)
    assertEquals( expected = 100L, actual = result[0].first)
    assertEquals( expected = 300L, actual = result[0].second)
}
```

Рисунок 4.1 — Модульный тест для проверки слияния интервалов

В качестве метрик, которые используются для оценки качества модульного тестирования, часто применяется расчет покрытия исходного кода существующими тестами. Под покрытием подразумевается доля объектов исходного кода, работа которых проверяется в ходе запуска всех тестов. Объектами исходного кода в случае с JVM-языками программирования являются: классы, методы, ветви, строки, инструкции (от крупного уровня к мелкому). Традиционно используется метрика покрытия строк исходного кода, в современной разработке наиболее полезной метрикой считается доля покрытия ветвей [27].

Для оценки покрытия применяются отдельные решения. В JVM-разработке наиболее популярным средством для оценки покрытия является JaCoCo. В случае использования языка Kotlin инструмент Kover от компании-разработчика языка (JetBrains) может оказаться лучшим решением за счет лучшей поддержки специализированных инструкций и кодогенерации Kotlin, которая не учитывается JaCoCo, особенно в старых версиях. По этой причине в работе над проектом использовался Kover. Результаты проверки покрытия могут быть представлены в различных видах; наиболее распространенными являются: HTML представление для просмотра человеком и xml-отчет для автоматизированных проверок (например, при проверке в конвейере доставки программного обеспечения).

Базовым уровнем для метрик покрытия строк и ветвлений считаются показатели в 80% и 70% соответственно. При написании тестов была поставлена цель достичь показателей в 90% и 80% процентов покрытия в каждом из разрабатываемых модулей.

Рассмотрим уровень покрытия на примере одного из самых важных модулей – модуле обработки телеметрических данных. Его задача – обрабатывать телеметрические данные и формировать события, отслеживая нахождение ТС в географических зонах. Код, определяющий факт вхождения в зоны, неустойчив к вносимым изменениям: его логика может быть легко

нарушена. Ввиду этого автоматизированные проверки работы пространственного индекса очень важны для сохранения корректной работы модуля и системы в целом.

Уровень покрытия тестами модуля обработки телеметрических данных показан на рисунке 4.2.

telemetry-processor: Overall Coverage Summary

Package	Class, %	Method, %	Branch, %	Line, %	Instruction, %
all classes	91,2% (31/34)	87% (100/115)	80,5% (169/210)	95,2% (668/702)	94,3% (3190/3384)

Coverage Breakdown

Package	Class, %	Method, %	Branch, %	Line, %	Instruction, %
com.lynceus.telemetry_processor.repository	100% (1/1)	100% (4/4)	100% (4/4)	100% (48/48)	100% (218/218)
com.lynceus.telemetry_processor.listener	100% (1/1)	100% (2/2)		100% (4/4)	100% (13/13)
com.lynceus.telemetry_processor.event	100% (4/4)	100% (4/4)		100% (23/23)	100% (124/124)
com.lynceus.telemetry_processor.entity	100% (1/1)	100% (4/4)	100% (10/10)	100% (32/32)	100% (145/145)
com.lynceus.telemetry_processor.config	100% (6/6)	100% (14/14)		100% (46/46)	100% (209/209)
com.lynceus.telemetry_processor.service	80% (4/5)	90,9% (20/22)	83,3% (35/42)	99,3% (134/135)	99,4% (695/699)
com.lynceus.telemetry_processor.processor	100% (5/5)	77,8% (14/18)	83,7% (77/92)	96,2% (231/240)	96,2% (1116/1160)
com.lynceus.telemetry_processor.dto	100% (5/5)	92% (23/25)		95,6% (43/45)	96,4% (106/110)
com.lynceus.telemetry_processor.handler	100% (2/2)	86,7% (13/15)	69,4% (43/62)	86% (104/121)	83,5% (557/667)
com.lynceus.telemetry_processor.controller	100% (1/1)	33,3% (1/3)		50% (2/4)	38,5% (5/13)
com.lynceus.telemetry_processor	33,3% (1/3)	25% (1/4)		25% (1/4)	7,7% (2/26)

Рисунок 4.2 — Уровень покрытия исходного кода тестами

Значения метрик покрытия строк и ветвлений превысил установленные значения, что позволяет сделать вывод о достижении необходимого уровня тестирования модуля.

Для остальных модулей системы также выполнено автоматизированное модульное тестирование, удовлетворяющее установленным уровням метрик.

4.1.2 Ручное функциональное тестирование

Ручное функциональное тестирование выполняется вручную с целью установления соответствия программного обеспечения заданным функциональным требованиям. Для выполнения ручного тестирования составляется список требований, которые необходимо проверить, в ходе тестирования описывается ожидаемый и фактический результат.

В ходе тестирования разрабатываемого продукта ручное тестирование также является системным, поскольку выполнение всех функциональных требований возможно только при работе всех её модулей ввиду микросервисного архитектурного подхода.

Перечень групп функциональных требований (установленных в п. 1.2.1), описания проверок при выполнении ручного тестирования, ожидаемые и фактические результаты отражены в таблице 4.1.

Таблица 4.1 — Результаты ручного функционального тестирования

№	Группа требований	Описание проверки	Ожидаемый результат	Фактический результат
1	Приём телеметрических данных	При переходе в пункт меню «ТС на карте» отображается позиция ТС на карте.	На карте отображается значок с текущим положением ТС, перемещаясь при поступлении новых данных.	На карте отображается метка (треугольник), показывающая местоположение ТС в соответствии с наиболее актуальными телеметрическими данными.
2		Данные отображаются корректно, метаданные доступны для просмотра.	Положение и направление метки соответствует телеметрическим данным. Доступен просмотр данных.	Положение метки соответствует географическим координатам. Направление метки соответствует азимуту (курсу). При наведении на метку появляется всплывающее окно с основными телеметрическими данными.
3		Проверка сохранения порядка приема телеметрических данных	Метка ТС перемещается, обновляется в порядке поступления данных	Метка ТС перемещается, сохраняя порядок поступления телеметрических данных. Дублирования метки не происходит. Резких прыжков метки не происходит.
4	Первичная обработка и хранение телеметрии	Проверка запроса исторических телеметрических данных для ТС.	В меню «Поиск навигации» на вкладке «Навигация устройства» доступен ввод фильтров и просмотр телеметрических данных.	Доступен ввод фильтров и запрос данных. Данные отображаются на карте в виде меток (направленных по курсу ТС треугольников), соединенных кривой линией. При наведении указателем мыши на метку появляется всплывающее окно с телеметрическими данными.

Продолжение таблицы 4.1

№	Группа требований	Описание проверки	Ожидаемый результат	Фактический результат
5	Зональное отслеживание в реальном времени	Проверка ведения справочника зон.	Географические зоны доступны для просмотра, создания и редактирования в меню «Зоны». Создание и редактирование выполняется успешно.	В меню «Зоны» на карте отображаются географические зоны. При нажатии на зону правой кнопкой мыши она открывается в редакторе. При нажатии на кнопку «Создать» открывается редактор для создания новой зоны. Сохранение/редактирование зоны выполняется успешно.
6	Зональное отслеживание в реальном времени	Проверка зонального потокового отслеживания	В меню «События» отображается поток событий зонального отслеживания. Показаны данные: ТС, вход/выход, время, зона. Генерируемые события соответствуют реальным.	При переходе в меню «События» показывается поток событий, близкий к реальному времени (в большинстве случаев задержка не более 1с относительно времени генерации пакета). Необходимые данные отображаются. При выборочной сверке событий с реальными данными расхождений не выявлено.
7	Пространственно-временной поиск телеметрии	Доступно начертание зоны для поиска телеметрических данных и ввод дискретизированных временных границ.	При нажатии на карту правой кнопкой мыши устанавливается точка полигона. После четырех нажатий отображаются данные выбранной области.	Полигон формируется согласно ожидаемому поведению. После установки четырех точек в интерфейсе показаны координаты нарисованного полигона.
8		Проверка ввода ограничений времени пространственно-временного поиска.	Устанавливается время, кратное 5 минутам.	При деактивации поля ввода значение в поле округляется до ближайшего значения, кратного пяти минутам.

Продолжение таблицы 4.1

№	Группа требований	Описание проверки	Ожидаемый результат	Фактический результат
9	Пространственно-временной поиск телеметрии	Проверяется корректность пространственно-временного поиска: ТС из результатов поиска действительно находились в указанном промежутке в указанной географической области.	При построении трека на карте наблюдается входение ТС в указанную область.	В результатах поиска рядом с каждым ТС имеется кнопка для запроса телеметрических данных за указанный период и отображение трека на карте. В результатах поиска присутствуют только те ТС, которые находились в указанной области в заданном временном промежутке.
10	Предоставление интерфейсов	Пользовательский графический интерфейс предоставляет все возможности для использования реализованных функций.	Интерфейс предоставляет визуализацию положения транспортных средств на карте, историю треков, конфигурирование геозон и просмотр событий.	В ходе предыдущих пунктов ручного тестирования наличие всех перечисленных требований к пользовательскому интерфейсу в конечном продукте было подтверждено.

Таким образом, в ходе ручного функционального тестирования было установлено соответствие программного продукта функциональным требованиям, определенным при постановке задачи.

4.1.3 Нагрузочное тестирование

Нагрузочное тестирование – тип нефункционального тестирования, которое оценивает поведение системы при разных уровнях нагрузки, в частности, определяет максимально допустимую нагрузку.

В качестве метрик производительности чаще всего выделяются следующие виды:

- время отклика;
- пропускная способность;
- загрузка ресурсов;
- количество ошибок.

Наиболее актуальной метрикой для оценки качества системы отслеживания транспортных средств представляется оценка пропускной

способности системы, поскольку природа предметной области и потоковая архитектура программного продукта накладывают высокие требования к количеству обрабатываемых сообщений за промежуток времени. Также можно сказать, что в производительности потоковой обработки сообщений заключается одна из основных ценностей программного продукта, ввиду сокращения расходов на вычислительные ресурсы относительно конкурентных решений.

Общая производительность стабильно и полноценно рабочей системы, выполняющей обмен данными с помощью брокера сообщений, определяется производительностью её наименее производительного модуля. Наиболее активный обмен данными происходит при приеме, обработке и хранении телеметрических данных. Поток телеметрических данных проходит от приемника телеметрии, который отправляет поток данных в брокер сообщений (Kafka). В качестве потребителей телеметрических данных из брокера сообщений выступают модули зонального отслеживания и обработки (сохранения) телеметрии. Таким образом, для тестирования всего контура обработки необходимо:

- протестировать каждый из трех компонентов контура отдельно;
- выявить наиболее узкое место – модуль, который определит максимальную нагрузку на систему.

Методика тестирования различается в зависимости от интерфейса взаимодействия модуля. Для приемной службы моделируется поток телеметрических данных с помощью отдельного тестового скрипта, фиксируется количество отправленных пакетов в отведенные промежутки времени. Для модулей, которые выполняют потребление телеметрических данных из брокера сообщений Kafka, тестирование выполняется следующим образом: в брокере накапливается массив данных (100 000 сообщений), затем модули запускаются по отдельности, замеряется время, за которое массив данных будет обработан. В каждом из перечисленных случаев выполняется несколько измерений, после

чего рассчитывается среднее значение, на основе которого делается заключение о производительности модуля.

Результаты нагрузочного тестирования представлены в таблице 4.2. Показатели приведены в сообщениях в секунду (с/с).

Таблица 4.2 — Результаты нагрузочного тестирования

№	Модуль	1, с/с	2, с/с	3, с/с	4, с/с	5, с/с	Среднее, с/с
1	Приемная служба	4657	4983	5143	4587	4872	4848,4
2	Зональное отслеживание	9870	10073	9732	9862	10024	9912,2
3	Сохранение телеметрии	3147	3209	3129	3065	3112	3132,4

Согласно результатам измерений, наименее узким местом обработки телеметрических данных является модуль сохранения телеметрии в базу данных. Это объясняется тем, что производительность баз данных, в особенности реляционных, ниже, чем производительность ОЗУ и сетевого обмена, а из тестируемых модулей модуль сохранения телеметрии использует БД в наибольшем объеме.

В соответствии с установленными нефункциональными требованиями, общая производительность системы не должна быть ниже 2000 сообщений в секунду. По результатам нагрузочного тестирования можно сделать вывод о том, что программный продукт соответствует нефункциональным требованиям о производительности.

4.2 Ввод в эксплуатацию

Развертывание программного обеспечения выполняется в среде изоляции Docker [21] с помощью описания развертывания компонентов с помощью конфигурационного файла Docker Compose. Такой способ развертывания

гарантирует максимальную независимость работы программной системы от окружения, поскольку единственным, что не контролируется при развертывании, является ядро ОС, на которой выполняется развертывание – остальные компоненты, такие как системный менеджер, доступные пакеты, сетевые настройки, переменные среды и прочее полностью контролируются конфигурационными файлами.

К системе, в которой выполняется запуск Docker-контейнеров, имеются следующие требования:

- ядро Linux – не ниже 3.10, рекомендуется не ниже 4.x, а лучше всего 5.6 (в т.ч. WSL в системе Windows);
- 64-битная операционная система;
- центральный процессор архитектуры x86_64 либо ARM64.

Конфигурация развертывания составляется в форматах YAML либо JSON. Ниже приведен листинг конфигурационного файла всех модулей системы в формате YAML.

```
services:
# ===== База данных =====
db:
  image: postgres:17-alpine
  container_name: postgres_db
  restart: always
  environment:
    POSTGRES_USER: ***
    POSTGRES_PASSWORD: ***
    POSTGRES_DB: lynceus
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data
  networks:
    - kafka-net

# ===== kafka =====
kafka:
  image: confluentinc/cp-kafka:latest
  container_name: kafka
  ports:
    - "9092:9092"
  networks:
    - kafka-net
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: 'broker,controller'
    KAFKA_CONTROLLER_LISTENER_NAMES: 'CONTROLLER'
    KAFKA_CONTROLLER_QUORUM_VOTERS: '1@kafka:9093'
```

```

    KAFKA_LISTENERS:
'INTERNAL://:29092,EXTERNAL://:9092,CONTROLLER://:9093'
    KAFKA_ADVERTISED_LISTENERS:
'INTERNAL://kafka:29092,EXTERNAL://localhost:9092'
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
'INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT'
    KAFKA_INTER_BROKER_LISTENER_NAME: 'INTERNAL'
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    CLUSTER_ID: 'Mku30EVBNTcwNTJENDM2Qk'

# ===== valkey (Redis) =====
valkey:
  image: valkey/valkey:latest
  container_name: valkey-server
  ports:
    - "6379:6379"
  command: valkey-server --notify-keyspace-events KEA
  restart: always
  networks:
    - kafka-net

# ===== Микросервисы (готовые образы) =====

# Пространственно-временной поиск
spatio-temporal:
  image: spatio-temporal:latest
  container_name: spatio-temporal
  restart: always
  ports:
    - "8081:8080"
  networks:
    - kafka-net

# Приём телеметрии
telemetry-receiver:
  image: telemetry-receiver:latest
  container_name: telemetry-receiver
  restart: always
  networks:
    - kafka-net
  environment:
    SPRING_PROFILES_ACTIVE: docker
    KAFKA_BOOTSTRAP_SERVERS: kafka:29092
    KAFKA_TOPIC_TELEMETRY_INPUT: telemetry.input
    SERVER_PORT: 8080

# Обработчик телеметрии (сохранение в БД)
telemetry-processor:
  image: telemetry-processor:latest
  container_name: telemetry-processor
  restart: always
  networks:
    - kafka-net
  environment:
    SPRING_PROFILES_ACTIVE: docker
    KAFKA_BOOTSTRAP_SERVERS: kafka:29092
    KAFKA_TOPIC_CONSUMER: telemetry.raw
    KAFKA_TOPIC_PROCESSED: telemetry.processed
    DB_HOST: db
    DB_PORT: 5432
    DB_NAME: lynceus
    DB_USER: ***
    DB_PASSWORD: ***

```

```

# Зональный трекер
zone-tracker:
  image: zone-tracker:latest
  container_name: zone-tracker
  restart: always
  networks:
    - kafka-net
  environment:
    SPRING_PROFILES_ACTIVE: docker
    KAFKA_BOOTSTRAP_SERVERS: kafka:29092
    KAFKA_TOPIC_CONSUMER: telemetry.processed
    KAFKA_TOPIC_ZONE_EVENTS: zone.events
    DB_HOST: db
    DB_PORT: 5432
    DB_NAME: lynceus
    DB_USER: user
    DB_PASSWORD: password
    REDIS_HOST: valkey
    REDIS_PORT: 6379

# API Gateway
api-gateway:
  image: api-gateway:latest
  container_name: api-gateway
  restart: always
  ports:
    - "8082:8080"
  networks:
    - kafka-net
  environment:
    SPRING_PROFILES_ACTIVE: docker
    TELEMETRY_RECEIVER_URL: http://telemetry-receiver:8080
    ZONE_TRACKER_URL: http://zone-tracker:8080
    DB_HOST: db
    DB_PORT: 5432
    DB_NAME: lynceus
    DB_USER: user
    DB_PASSWORD: password

# web UI (фронтенд)
web-ui:
  image: web-ui:latest
  container_name: web-ui
  restart: always
  ports:
    - "80:80"
  networks:
    - kafka-net
  environment:
    API_BASE_URL: http://api-gateway:8082

networks:
  kafka-net:
    driver: bridge

volumes:
  postgres_data:

```

Приведенная конфигурация сохраняется в файл `docker-compose.yaml`, после чего запускается консольной командой в контексте директории с файлом:

```
docker compose up -d
```

После чего выполняется загрузка образов, создание контейнеров, их конфигурирование и запуск.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы выполнены этапы проектирования, реализации, тестирования и ввода в эксплуатацию программного продукта.

Проведен анализ предметной области: описано текущее состояние предметной области, определены требования к продукту, выполнено сравнение с аналогами, показана целесообразность разработки программного продукта.

Выполнено проектирование продукта: определены технические и программные средства разработки, тестирования и развертывания ПО.

На этапе реализации описана архитектура системы, приведено описание каждой из подсистем, их назначение, формат обмена данными и алгоритмы их работы. Указано описание структур данных. Описан в текстовом и графическом виде пользовательский графический интерфейс.

Проведено автоматизированное модульное, ручное функциональное и нагрузочное (нефункциональное) тестирование. Показано соответствие программного продукта установленным требованиям.

Выполнено развертывание системы, приведены системные требования и конфигурационные файлы.

Разработанная система представляет из себя систему мониторинга транспортных средств с обработкой событий в реальном времени. В ходе работы системы выполняется потоковая обработка телеметрических данных с помощью технических средств:

1. брокера сообщений Apache Kafka, обеспечивающего немедленную доставку данных от поставщика к потребителю;
2. хранилища «ключ-значение» Redis с поддержкой подписки на изменение данных по произвольному ключу;
3. библиотеки географической индексации Google S2 Geometry Library, предоставляющей возможность построения пространственных индексов.

Применение этих и других решений позволило создать систему, способную доставлять результаты обработки телеметрической информации заинтересованным лицам в кратчайшие сроки.

Конкурентными преимуществами разработанного продукта являются:

1. позиционное отслеживание на базе географических зон с использованием пространственного высокоэффективного индекса собственной разработки (на базе S2);
2. пространственно-временной поиск исторических телеметрических данных;
3. возможность горизонтального масштабирования, определенная возможностями расширения кластера Apache Kafka и объемом вычислительных мощностей предприятия.

Дальнейшее развитие системы предполагается в направлениях:

1. создания собственного приемника и ретранслятора телеметрических данных по различным протоколам;
2. разработки интеграционного API для взаимодействия со стороны внешних систем;
3. добавления модуля для анализа движений ТС и выгрузки отчетов (пробег, нарушения, зональное отслеживание);
4. поддержки авторизации и аутентификации, разграничения прав пользователей.

Задачи работы выполнены, цель достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Росстат - Транспорт // Росстат URL: <https://rosstat.gov.ru/statistics/transport> (дата обращения: 03.05.2026).
2. Постановление Правительства Российской Федерации "Об утверждении Правил оснащения транспортных средств категорий М2, М3 и транспортных средств категории N, используемых для перевозки опасных грузов, аппаратурой спутниковой навигации" от 22.12.2020 № 2216 // Официальное опубликование правовых актов. 2020 г. № 0001202012250015.
3. Постановление Правительства Российской Федерации "О взимании платы в счет возмещения вреда, причиняемого автомобильным дорогам общего пользования федерального значения транспортными средствами, имеющими разрешенную максимальную массу свыше 12 тонн" от 14.06.2013 № 504 // Официальное опубликование правовых актов. 2013 г. № 0001201306180005.
4. Зиарманд Артур Нисарович, Хаханов Владимир Иванович. Модели и методы мониторинга и управления транспортом // Радиоэлектроника и информатика. 2016. №3.
5. Игумнов Артем Олегович Архитектура программного обеспечения распределенной системы мониторинга и управления транспортом // Доклады ТУСУР. 2021. №2.
6. Кушнир Надежда Владимировна, Вонарх Юлия Сергеевна, Деев Илья Максимович, Долунц Анастасия Станиславовна, Карпенко Никита Андреевич Анализ данных в реальном времени: преимущества и примеры использования stream processing // Научный журнал КубГАУ. 2025. №207.
7. Баженов Артём Эдуардович, Райков Александр Вячеславович, Нехаев Максим Вадимович, Ореховский Никита Викторович. Оптимизация взаимодействия микросервисов с использованием асинхронных вызовов и брокеров сообщений (Kafka, RabbitMQ) // Программные системы и вычислительные методы. 2025. №4.

8. Тюкачев Николай Аркадиевич Алгоритм определения принадлежности точки многоугольнику общего вида или многограннику с треугольными гранями // Вестник ТГТУ. 2009. №3.

9. Долгий П. С. ПРОГРАММНЫЕ ОСОБЕННОСТИ РАЗРАБОТКИ ГИС-ПРОЕКТА «ГЕОДИНАМИКА И ТЕХНОГЕНЕЗ БЕЛАРУСИ» // Вестник Полоцкого государственного университета. Серия F. Строительство. Прикладные науки. 2023. №1.

10. Darius Šidlauskas, Simonas Šaltenis, Christian W. Christiansen, Jan M. Johansen, and Donatas Šaulys. Trees or grids? indexing moving objects in main memory. // In Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS '09). Association for Computing Machinery, New York, NY, USA, 236–245. 2009.

11. Geospatial Grid System // Microsoft Learn URL: <https://learn.microsoft.com/ru-ru/kusto/query/geospatial-grid-systems?view=azure-data-explorer> (дата обращения: 03.05.2026).

12. Al-Nsour, Esam & Sleit, Azzam & Alshraideh, Mohammad. A Pictorial Performance Comparison of Spatial Indexes. // 11th International Conference on Information and Communication Systems (ICICS) 042-047. 2020.

13. Толмачев Павел Владимирович PostgreSQL 16. Оптимизация запросов // Москва, Постгрес Профессиональный, 2026.

14. П. И. Смирнов, М. Ю. Карелина, Б. С. Субботин. Применение систем транспортной телематики для повышения эффективности эксплуатации техники // Министерство науки и высшего образования Российской Федерации, Вологодский государственный университет. – Вологда: ВоГУ, 2023. с. 123-124

15. JIANG Bohui, ZHOU Weifeng. Comparative Analysis of GeoHash, Google S2 and Uber H3 as Global Geographic Grid Coding Methods // Geography & Geographic Information Science. 2024. №40.

16. Marcin Gorawski, Adam Dyga. Indexing of spatio-temporal telemetric data based on distributed mobile bucket index // Proceedings of the 24th IASTED

international conference on Parallel and distributed computing and networks. 2006. С. 292-297.

17. ГОСТ 33465-2015. Глобальная навигационная спутниковая система. Система экстренного реагирования при авариях. Протокол обмена данными устройства/системы вызова экстренных оперативных служб с инфраструктурой системы экстренного реагирования при авариях // Москва: Стандартинформ, 2017. 65 с.

18. Performance Impact of 10,000+ Geofences in Traccar // Traccar URL: <https://www.traccar.org/forums/topic/performance-impact-of-10000-geofences-in-traccar/> (дата обращения: 03.05.2026).

19. Ограничения // Wialon Справочный центр URL: <https://help.wialon.com/ru/wialon-local/2304/user-guide/monitoring-system/limitations> (дата обращения: 03.05.2026).

20. Системные требования // Fort Monitor URL: <https://support.fort-monitor.ru/article/30936> (дата обращения: 03.05.2026).

21. Кусепова Л.Т., Кусепова Г.Т. Анализ и сравнение инструментов оркестрации контейнеров в локальной и облачной инфраструктуре // Вестник Казахстанско-Британского технического университета. 2026;23(1):22-36.

22. Баланов А.Н. Построение микросервисной архитектуры и разработка высоконагруженных приложений. Учебное пособие // М. СПб. Краснодар: Лань, 2024.

23. Оптимизация Redis для высоких нагрузок: полное руководство // Habr URL: <https://habr.com/ru/companies/selectel/articles/931760/> (дата обращения: 23.05.2026).

24. Чакон Скотт, Штрауб Бен Pro Git на русском. Всё, что нужно знать о Git // 2026. 541 с.

25. Webpack, Vite или Rspack: что это за зоопарк или чем собирать микрофронтенды в 2025? // Habr URL: <https://habr.com/ru/articles/888478/> (дата обращения: 23.05.2026).

26. Пышкин, Евгений Валерьевич. Модульное тестирование программного обеспечения [Текст] : профессиональный базовый курс с практикой на JUnit // СПб., АйТи-Подготовка, 2015. – 239 с.

27. How to make sense of Code Coverage metrics // Microsoft Learn URL: <https://learn.microsoft.com/sr-cyrl-rs/archive/blogs/jamesnewkirk/how-to-make-sense-of-code-coverage-metrics> (дата обращения: 24.05.2026).

ПРИЛОЖЕНИЕ А Системные требования

Минимальные системные требования для развертывания всех модулей системы при нагрузке, не превышающей 2000 сообщений в секунду:

- Процессор (CPU): Intel Core i7 (3-го поколения) или аналог, частота от 3.4 ГГц (4 ядра / 8 потоков);
- Оперативная память (RAM): от 8 ГБ (DDR3 и выше);
- Диск: SSD от 100 ГБ свободного места;
- Система: Linux (Ubuntu 22.04 LTS / Debian 11 / другая с поддержкой Docker) или Windows 10/11 с WSL2;
- Интернет: от 100 Мбит/с (получение телеметрических данных).

ПРИЛОЖЕНИЕ Б Исходный код

1. Исходный код пространственного индекса: S2ZoneIndex.kt

```
package com.lynceus.telemetry_processor.processor

import com.google.common.geometry.S2CellId
import com.google.common.geometry.S2Point
import com.google.common.geometry.S2Region
import com.google.common.geometry.S2RegionCoverer
import com.google.common.geometry.primitives.IntVector

class S2ZoneIndex(
    val regions: List<S2Region>,
    coverer: S2RegionCoverer,
    private val targetLevel: Int
) {
    private val intervalStarts: LongArray
    private val intervalEnds: LongArray
    private val regionIndicesPerInterval: Array<IntArray>

    // Вспомогательный класс для событий заметания
    private class SweepEvent(
        val point: Long,
        val regionIdx: Int,
        val isStart: Boolean
    )

    init {
        val localCoverer = S2RegionCoverer.builder()
            .setMaxCells(coverer.maxCells())
            .setMinLevel(coverer.minLevel())
            .setMaxLevel(targetLevel)
            .setLevelMod(coverer.levelMod())
            .build()

        // Сырые сегменты одного региона (для локального сжатия)
        class RawSegment(val start: Long, var end: Long, val regionIdx: Int)
        val initialSegments = mutableListOf<RawSegment>()

        // ШАГ 1: Получение покрытий и локальное слияние смежных ячеек для каждого региона отдельно
        for ((idx, region) in regions.withIndex()) {
            val covering = localCoverer.getCovering(region)
            var prevSeg: RawSegment? = null

            for (cellId in covering.cellIds()) {
                val start = cellId.childBegin(targetLevel).id()
                val end = cellId.childEnd(targetLevel).id()

                if (prevSeg != null && prevSeg.end == start) {
                    prevSeg.end = end // Сливаем смежные участки
                } else {
                    val newSeg = RawSegment(start, end, idx)
                    initialSegments.add(newSeg)
                    prevSeg = newSeg
                }
            }
        }

        // ШАГ 2: Создание списка событий для Sweep-line
        val events = ArrayList<SweepEvent>(initialSegments.size * 2)
        for (seg in initialSegments) {
            events.add(SweepEvent(seg.start, seg.regionIdx, true))
            events.add(SweepEvent(seg.end, seg.regionIdx, false))
        }
        // Сортировка: O(M log M)
        events.sortWith(
            compareBy<SweepEvent> { it.point }
                .thenByDescending { it.isStart }
        )

        val groupedEvents = events.groupBy { it.point }

        // Списки для сборки финальных интервалов
        val startsList = mutableListOf<Long>()
```

```

val endsList = mutableListOf<Long>()
val regionsList = mutableListOf<IntArray>()

val activeRegions = hashSetOf<Int>()
var lastPoint = -1L

// идем по точкам пространства
for ((point, events) in groupedEvents) {

    // добавляем участок, если на нем есть интервалы
    if (lastPoint != -1L &&
        activeRegions.isNotEmpty()) {
        startsList.add(lastPoint)
        endsList.add(point)
        regionsList.add(activeRegions.sorted().toIntArray())
    }

    // помечаем новые активные зоны
    for (event in events) {
        if (event.isStart) {
            activeRegions += event.regionIdx
        }
        else {
            activeRegions -= event.regionIdx
        }
    }

    lastPoint = point
}

// Переносим результат в финальные плоские массивы
intervalStarts = startsList.toLongArray()
intervalEnds = endsList.toLongArray()
regionIndicesPerInterval = regionsList.toTypedArray()
}

// вектор для хранения результатов при поиске регионов,
// чтобы избежать аллокации массива при каждом вызове метода
private val resultVector = IntVector()

fun findRegions(p: S2Point): IntArray {
    val id = S2CellId.fromPoint(p)
        .parent(targetLevel).id()

    // инициализация окна бинарного поиска
    var low = 0
    var high = intervalStarts.size - 1
    var idx = -1

    // бинарный поиск
    while (low <= high) {
        // срединное значение окна поиска = (start + end) / 2
        val mid = (low + high) ushr 1

        // достаем начало и конец
        val start = intervalStarts[mid]
        val end = intervalEnds[mid]

        when {
            id < start -> high = mid - 1 // id до середины - берем меньшую половину
            id >= end -> low = mid + 1 // id после середины - берем большую половину
            else -> {
                idx = mid // нашли результат
                break
            }
        }
    }

    if (idx == -1) return EMPTY_INT_ARRAY

    val candidates = regionIndicesPerInterval[idx]

    resultVector.clear()

    for (i in candidates.indices) {

```

```

        val rIdx = candidates[i]
        if (regions[rIdx].contains(p)) {
            resultVector.add(rIdx)
        }
    }

    return if (resultVector.size() == candidates.size)
        candidates
    else
        resultVector.toArray()
}

companion object {
    private val EMPTY_INT_ARRAY = IntArray(0)
}
}

```

2. Обработчик событий входа/выхода из географических зон

ZoneVisitEventProcessor.kt

```

package com.lynceus.telemetry_processor.processor

import com.google.common.geometry.*
import com.lynceus.telemetry_processor.dto.GeozoneDto
import com.lynceus.telemetry_processor.entity.TelemetryPacket
import com.lynceus.telemetry_processor.event.InOut
import com.lynceus.telemetry_processor.event.ZoneVisitEvent
import com.lynceus.telemetry_processor.service.GeozoneService
import org.springframework.context.ApplicationEventPublisher
import org.springframework.stereotype.Component

@Component
class ZoneVisitEventProcessor(
    private val geozoneService: GeozoneService,
    private val applicationEventPublisher: ApplicationEventPublisher
) {
    private val zoneIdToZones: Map<Long, GeozoneDto>
    private val deviceToCurrentZoneIdSet = hashMapOf<Long, HashSet<Long>>()
    private val zoneIndex: S2ZoneIndex
    private val zones: List<GeozoneDto>

    init {
        // все зоны грузим в пространственный индекс
        zones = geozoneService.getAllGeozones()
        val shapeToZones = hashMapOf<S2Shape, MutableList<GeozoneDto>>()

        zoneIdToZones = zones.associateBy { it.id }

        var validCount = 0
        var totalCount = 0
        val regions = mutableListOf<S2Region>()

        for (zone in zones) {
            totalCount++
            val points = zone.coordinates.map { S2LatLng.fromDegrees(
                it[0],
                it[1]
            ) }.toPointList()

            val loop = S2Loop(points)
            loop.normalize()

            // Check loop validity
            if (!loop.isValid) {
                println("DEBUG: Loop is invalid for zone ${zone.id}")
                continue
            }

            val polygon = S2Polygon(loop)

            if (polygon.isValid) {
                validCount++
                regions.add(polygon)
            } else {
                // Polygon is invalid, skip it
            }
        }
    }
}

```

```

        println("DEBUG: Polygon is invalid for zone ${zone.id}, loop valid=${loop.isValid}")
    }
}

// Debug output
println("ZoneVisitEventProcessor init: $validCount/$totalCount polygons valid")

zoneIndex = S2ZoneIndex(
    regions = regions,
    coverer = S2RegionCoverer.builder()
        .setMaxLevel(24)
        .setMinLevel(1)
        .setMaxCells(200)
        .build(),
    targetLevel = 24
)

fun processPacket(packet: TelemetryPacket) {

    val point = S2LatLng.fromDegrees(
        packet.latitude,
        packet.longitude).toPoint()
    val regions = zoneIndex.findRegions(point)
    val oldZoneIds = deviceToCurrentZoneIdSet.computeIfAbsent(
        packet.deviceId) {
            hashSetOf()
        }
    var newZoneIds: HashSet<Long>? = null
    val zones = regions.map { zones[it] }

    for (region in zones) {

        if (zones != null) {
            newZoneIds = newZoneIds ?: hashSetOf()
            for (zone in zones) {
                newZoneIds.add(zone.id)
            }
        }

    }

    val newZones = newZoneIds ?: emptySet()

    val addedZoneIds = newZones.subtract(oldZoneIds)
    val removedZoneIds = oldZoneIds.subtract(newZones)

    // ВХОД В ЗОНУ
    for (addedZoneId in addedZoneIds) {
        val zone = zoneIdToZones[addedZoneId]

        if (zone != null) {
            publishEvent(zone, packet, InOut.In)
        }
    }

    // ВЫХОД ИЗ ЗОНЫ
    for (removedZoneId in removedZoneIds) {
        val zone = zoneIdToZones[removedZoneId]

        if (zone != null) {
            publishEvent(zone, packet, InOut.Out)
        }
    }

    if (newZoneIds != null) {
        deviceToCurrentZoneIdSet[packet.deviceId] = newZoneIds
    }
    else {
        deviceToCurrentZoneIdSet.remove(packet.deviceId)
    }
}

fun publishEvent(zone: GeozoneDto, packet: TelemetryPacket, inOut: InOut) {
    val event = ZoneVisitEvent(
        inOut = inOut,
        deviceId = packet.deviceId,
        vehicleId = packet.vehicleId,

```



```

        zoneId = zone.id,
        zoneName = zone.name,
        zoneDateTime = packet.packetTime
    )
    applicationEventPublisher.publishEvent(event)
}
}

```

3. Обработчик пользовательских соединений для трансляции телеметрических данных TelemetryWebSocketHandler.kt

```

package com.lynceus.telemetry_processor.handler

import com.fasterxml.jackson.databind.ObjectMapper
import com.google.common.geometry.S2ContainsPointQuery
import com.google.common.geometry.S2LatLng
import com.google.common.geometry.S2Loop
import com.google.common.geometry.S2Polygon
import com.lynceus.telemetry_processor.entity.TelemetryPacket
import org.slf4j.LoggerFactory
import org.springframework.data.redis.connection.MessageListener
import org.springframework.data.redis.core.RedisTemplate
import org.springframework.data.redis.core.ScanOptions
import org.springframework.data.redis.listener.PatternTopic
import org.springframework.data.redis.listener.RedisMessageListenerContainer
import org.springframework.stereotype.Component
import org.springframework.web.socket.CloseStatus
import org.springframework.web.socket.TextMessage
import org.springframework.web.socket.WebSocketSession
import org.springframework.web.socket.handler.TextWebSocketHandler
import java.util.concurrent.ConcurrentHashMap
import com.google.common.geometry.S2ShapeIndex

@Component
class TelemetryWebSocketHandler(
    private val redisTemplate: RedisTemplate<String, Any>,
    private val objectMapper: ObjectMapper,
    private val redisMessageListenerContainer: RedisMessageListenerContainer
) : TextWebSocketHandler() {

    private val logger = LoggerFactory.getLogger(this.javaClass)

    // Сессии, подписанные на определённые ячейки
    private val sessionCellSubscriptions = ConcurrentHashMap<WebSocketSession,
    S2ContainsPointQuery>()

    // Обработчик сообщений из Redis keyspace notifications
    private val redisMessageListener = MessageListener { message, pattern ->

        if (pattern != null) {
            handleKeyspaceNotification(
                String(message.body),
                String(pattern)
            )
        }
    }

    // Pattern для подписки на set операции ключей nav:*.last
    companion object {
        const val KEYSPACE_PATTERN = "__keyevent@0__:set"
        const val NAV_KEY_PREFIX = "nav:"
        const val NAV_KEY_SUFFIX = ":last"
    }

    init {
        // Подписываемся на keyspace notifications для set операций
        redisMessageListenerContainer.addMessageListener(redisMessageListener,
        PatternTopic(KEYSPACE_PATTERN))
        logger.info("Redis keyspace notification listener registered for pattern:
        $KEYSPACE_PATTERN")
    }

    override fun afterConnectionEstablished(session: WebSocketSession) {

```

```

        logger.info("WebSocket connection established: ${session.id}")
    }

    override fun handleTextMessage(session: WebSocketSession, message: TextMessage) {
        try {
            val payload = message.payload
            logger.debug("Received message from ${session.id}: $payload")

            val request = objectMapper.readTree(payload)
            val typeNode = request["type"]
            if (typeNode == null || typeNode.isNull) {
                session.sendMessage(TextMessage("""{"error": "Missing 'type' field"}"""))
                return
            }
            val type = typeNode.asText()
            if (type == null || type.isBlank()) {
                session.sendMessage(TextMessage("""{"error": "Missing 'type' field"}"""))
                return
            }

            when (type) {
                "subscribe_polygon" -> handleSubscribePolygon(session, request)
                else -> session.sendMessage(TextMessage("""{"error": "Unknown message type:
$type"}"""))
            }
        } catch (e: Exception) {
            logger.error("Error processing WebSocket message", e)
            session.sendMessage(TextMessage("""{"error": "${e.message}"}"""))
        }
    }

    private fun handleSubscribePolygon(session: WebSocketSession, request:
com.fasterxml.jackson.databind.JsonNode) {
        val pointsNode = request["points"]
        if (pointsNode == null || pointsNode.isNull) {
            session.sendMessage(TextMessage("""{"error": "Missing 'points' field"}"""))
            return
        }
        if (!pointsNode.isArray) {
            session.sendMessage(TextMessage("""{"error": "Points must be an array"}"""))
            return
        }
        if (pointsNode.size() < 3) {
            session.sendMessage(TextMessage("""{"error": "Polygon must have at least 3 points"}"""))
            return
        }

        val points = mutableListOf<Pair<Double, Double>>()
        for (point in pointsNode) {
            val lat = point["lat"].asDouble()
            val lon = point["lon"].asDouble()
            points.add(lat to lon)
        }

        val s2Points = points.map { S2LatLng.fromDegrees(
            it.first,
            it.second).toPoint() }
        val loop = S2Loop(s2Points)
        val polygon = S2Polygon(loop)

        val index = S2ShapeIndex()
        index.add(polygon.shape())

        // Создаем объект запроса
        // Reuse этого объекта важен для производительности (кэширование)
        val query = S2ContainsPointQuery(index)

        // Очищаем предыдущие подписки сессии
        sessionCellSubscriptions.remove(session)

        // новая подписка
        sessionCellSubscriptions[session] = query

        // Для каждой ячейки получаем устройства
        val allDeviceIds = mutableSetOf<Long>()
        val options = ScanOptions.scanOptions()
            .match("nav:*:last") // Задаем паттерн (например, "myKey*")
    }

```

```

        .build()

// ищем все ключи навигации
val cursor = redisTemplate.scan(options)
val keys = mutableListOf<String>()

if (cursor != null) {
    while (cursor.hasNext()) {
        val key = cursor.next()
        keys.add(key)
    }
}

// берем всю навигацию
val data = if (keys.isNotEmpty()) redisTemplate.opsForValue().multiGet(keys) else null

// если навигация входит в точку, отдаем клиенту (должно быть быстро)
if (data != null) {
    for (value in data) {
        if (value != null) {
            val str = value.toString()
            try {
                val obj = objectMapper.readValue(str, TelemetryPacket::class.java)
                val s2Point = S2LatLng.fromDegrees(obj.latitude, obj.longitude).toPoint()

                if (query.contains(s2Point)) {
                    session.sendMessage(TextMessage(str))
                }
            } catch (e: Exception) {
                logger.error("Failed to parse telemetry data: $str", e)
            }
        }
    }
}

session.sendMessage(TextMessage("""{"status": "subscribed"}"""))
}

@Suppress("UNUSED_PARAMETER")
fun handleKeyspaceNotification(message: String, pattern: String) {
    //logger.info("Received Redis keyspace notification on pattern $pattern: $message")

    if (sessionCellSubscriptions.isEmpty()) {
        return
    }

    // message содержит ключ, который был изменен (например "nav:123:last")
    if (!message.startsWith(NAV_KEY_PREFIX) || !message.endsWith(NAV_KEY_SUFFIX)) {
        return // нас интересуют только ключи nav:*.last
    }

    try {
        // Извлекаем deviceId из ключа
        val deviceId = extractDeviceIdFromKey(message)
        if (deviceId == null) {
            logger.warn("Failed to extract deviceId from key: $message")
            return
        }

        // Получаем обновленные данные из Redis
        val data = redisTemplate.opsForValue().get(message) as? String
        if (data == null) {
            logger.warn("No telemetry data found for key: $message")
            return
        }

        val telemetryPacket = objectMapper.readValue(
            data,
            TelemetryPacket::class.java
        )
        val s2Point = S2LatLng.fromDegrees(
            telemetryPacket.latitude,
            telemetryPacket.longitude
        ).toPoint()

        // Находим сессии, подписанные на эту ячейку
        val sessions = sessionCellSubscriptions.filter { it.value.contains(s2Point) }.keys
        if (sessions.isEmpty()) {
            //logger.debug("No sessions subscribed to cell $s2Cell for device $deviceId")
        }
    }
}

```

```

        return
    }

    //logger.debug("Sending update to ${sessions.size} sessions for device $deviceId in cell
    $s2Cell")

    // Отправляем обновление всем подписанным сессиям
    for (session in sessions) {
        if (session.isOpen) {
            try {
                session.sendMessage(TextMessage(data))
            } catch (e: Exception) {
                logger.error("Failed to send telemetry update to session ${session.id}", e)
            }
        }
    }
} catch (e: Exception) {
    logger.error("Failed to process keyspace notification for key: $message", e)
}

}

fun extractDeviceIdFromKey(key: String): Long? {
    // key format: "nav:{deviceId}:last"
    val regex = "^nav:(\\d+):last$".toRegex()
    return regex.find(key)?.groupValues?.get(1)?.toLongOrNull()
}

override fun afterConnectionClosed(session: WebSocketSession, status: CloseStatus) {
    logger.info("WebSocket connection closed: ${session.id}, status: $status")
    // Удаляем подписки сессии
    sessionCellSubscriptions.remove(session)
}

override fun handleTransportError(session: WebSocketSession, exception: Throwable) {
    logger.error("Transport error for session ${session.id}", exception)
}
}

```

4. Запрос телеметрических данных из БД по временным и пространственно-временным условиям TelemetryQueryService.kt

```

package com.lynceus.telemetry_processor.service

import com.google.common.geometry.S2LatLng
import com.google.common.geometry.S2Loop
import com.google.common.geometry.S2Polygon
import com.google.common.geometry.S2RegionCoverer
import com.lynceus.telemetry_processor.dto.DeviceTelemetryRequest
import com.lynceus.telemetry_processor.dto.PolygonTimeRequest
import com.lynceus.telemetry_processor.dto.TelemetryIntervalResponse
import com.lynceus.telemetry_processor.entity.TelemetryPacket
import com.lynceus.telemetry_processor.processor.NavigationProcessor
import com.lynceus.telemetry_processor.repository.TelemetryPacketRepository
import org.springframework.stereotype.Service
import java.time.LocalDateTime

@Service
class TelemetryQueryService(
    private val telemetryPacketRepository: TelemetryPacketRepository
) {

    fun findTelemetryIntervals(request: PolygonTimeRequest): List<TelemetryIntervalResponse> {

        val points = request.polygon.map { S2LatLng.fromDegrees(
            it.latitude,
            it.longitude).toPoint() }

        val s2Loop = S2Loop(points)
        s2Loop.normalize()
        val s2Polygon = S2Polygon(s2Loop)
    }
}

```

```

if (!s2Polygon.isValid) {
    return emptyList()
}

val coverer = S2RegionCoverer.builder()
    .setMaxLevel(NavigationProcessor.S2_ZONE_LEVEL) // Максимальный уровень
    .setMinLevel(1) // Минимальный уровень (самые большие ячейки)
    .setMaxCells(200) // Больше ячеек для лучшего покрытия
    .build()

// Получаем покрытие прямоугольника S2 ячейками разных уровней
val cellUnion = coverer.getCovering(s2Polygon)

val ranges = mutableListOf<Pair<Long, Long>>()

for (i in 0 until cellUnion.size()) {
    val cell = cellUnion.cellId(i)

    // Получаем диапазон дочерних ячеек уровня 24
    val rangeStart = cell.childBegin(24).id()
    val rangeEnd = cell.childEnd(24).id() - 1 // -1 потому что childEnd исключительный

    ranges.add(Pair(rangeStart, rangeEnd))
}

// Сортируем по начальному ID
ranges.sortBy { it.first }

val mergedRanges = mergeAdjacentRanges(ranges)

val foundDevices = telemetryPacketRepository.findByS2KeyInRanges(
    request.fromDateTime,
    request.toDateTime,
    mergedRanges
)

return foundDevices
}

fun getDeviceTelemetryInPeriod(request: DeviceTelemetryRequest): List<TelemetryPacket> {
    return telemetryPacketRepository.getDeviceTelemetryInPeriod(
        request.deviceId,
        request.fromDateTime,
        request.toDateTime
    )
}

private fun mergeAdjacentRanges(sortedRanges: List<Pair<Long, Long>>): List<Pair<Long, Long>> {
    if (sortedRanges.isEmpty()) {
        return emptyList()
    }

    val merged = mutableListOf<Pair<Long, Long>>()
    var currentStart = sortedRanges[0].first
    var currentEnd = sortedRanges[0].second

    for (i in 1 until sortedRanges.size) {
        val (nextStart, nextEnd) = sortedRanges[i]

        // Если интервалы пересекаются или соседние (разница в 1)
        if (nextStart <= currentEnd + 1) {
            // Объединяем
            currentEnd = maxOf(currentEnd, nextEnd)
        } else {
            // Сохраняем текущий и начинаем новый
            merged.add(Pair(currentStart, currentEnd))
            currentStart = nextStart
            currentEnd = nextEnd
        }
    }

    // Добавляем последний интервал
    merged.add(Pair(currentStart, currentEnd))

    return merged
}

```

```

    }
  }
}

```

5. Пользовательский интерфейс – отображение телеметрических данных на карте в реальном времени VehicleMap.vue

```

<template>
  <div class="vehicles-map-container">
    <div ref="mapContainer" class="vehicles-map"></div>
    <div v-if="connectionStatus !== 'OPEN'" class="connection-status">
      <div class="status-indicator" :class="connectionStatus.toLowerCase()"></div>
      <span class="status-text">{{ connectionStatusText }}</span>
    </div>
    <div v-if="vehicleCount > 0" class="vehicle-counter">
      Транспортных средств: {{ vehicleCount }}
    </div>
    <div
      v-if="popupVisible"
      class="vehicle-popup"
      :style="{ left: `${popupLeft}px`, top: `${popupTop}px` }"
      @mouseenter="cancelPopupHide"
      @mouseleave="schedulePopupHide"
    >
      <div class="popup-header">
        <span class="popup-title">Данные ТС</span>
        <button class="popup-close" @click="closePopup">×</button>
      </div>
      <div class="popup-content">
        <pre class="json-data">{{ popupData }}</pre>
      </div>
    </div>
  </div>
</template>

<script setup lang="ts">
import { ref, onMounted, onBeforeUnmount, watch } from 'vue'
import Map from 'ol/Map'
import View from 'ol/View'
import TileLayer from 'ol/layer/Tile'
import VectorLayer from 'ol/layer/Vector'
import VectorSource from 'ol/source/Vector'
import OSM from 'ol/source/OSM'
import { fromLonLat, toLonLat } from 'ol/proj'
import { getWebSocketUrl, TelemetryWebSocket, type Point } from '../api/telemetry'
import { MarkerManager } from '../utils/markerUtils'

const mapContainer = ref<HTMLDivElement | null>(null)
const map = ref<Map | null>(null)
let vectorSource: VectorSource | null = null
let vectorLayer: VectorLayer | null = null

// WebSocket connection
const ws = ref<TelemetryWebSocket | null>(null)
const connectionStatus = ref<'CONNECTING' | 'OPEN' | 'CLOSED' | 'ERROR'>('CONNECTING')
const connectionStatusText = ref('Подключение...')
const vehicleCount = ref(0)

// Marker management
const markerManager = ref<MarkerManager | null>(null)

// Popup variables (simple approach like ZoneMap.vue)
const popupVisible = ref(false)
const popupData = ref<string>('')
const popupLeft = ref(0)
const popupTop = ref(0)
let hidePopupTimer: number | null = null
const HIDE_POPUP_DELAY = 300 // ms

// Subscription management
let currentSubscription: Point[] | null = null
let subscriptionDebounceTimer: number | null = null

```

```

const SUBSCRIPTION_DEBOUNCE_MS = 500

function createMap() {
  if (!mapContainer.value) return

  // Create vector source and layer for vehicle markers
  vectorSource = new VectorSource()
  vectorLayer = new VectorLayer({
    source: vectorSource
  })

  map.value = new Map({
    target: mapContainer.value,
    layers: [
      new TileLayer({ source: new OSM() }),
      vectorLayer
    ],
    view: new View({
      center: fromLonLat([30.437888132963106, 59.962961568076395]), // Санкт-Петербург
      zoom: 12,
      minZoom: 3,
      maxZoom: 22
    }),
    controls: []
  })

  // Listen to map view changes to update subscription
  map.value.getView().on('change:center', debounceUpdateSubscription)
  map.value.getView().on('change:resolution', debounceUpdateSubscription)

  // Add pointermove listener for hover popup (like ZoneMap.vue)
  map.value.on('pointermove', handlePointerMove)
}

function debounceUpdateSubscription() {
  if (subscriptionDebounceTimer) {
    clearTimeout(subscriptionDebounceTimer)
  }

  subscriptionDebounceTimer = window.setTimeout(() => {
    updateSubscription()
    subscriptionDebounceTimer = null
  }, SUBSCRIPTION_DEBOUNCE_MS)
}

function getMapViewPolygon(): Point[] {
  if (!map.value) return []

  const view = map.value.getView()
  const size = map.value.getSize()
  if (!size) return []

  const extent = view.calculateExtent(size)

  // Convert extent corners to lon/lat
  const bottomLeft = toLonLat([extent[0], extent[1]])
  const bottomRight = toLonLat([extent[2], extent[1]])
  const topRight = toLonLat([extent[2], extent[3]])
  const topLeft = toLonLat([extent[0], extent[3]])

  // Return polygon points in clockwise order
  return [
    { lon: bottomLeft[0], lat: bottomLeft[1] },
    { lon: bottomRight[0], lat: bottomRight[1] },
    { lon: topRight[0], lat: topRight[1] },
    { lon: topLeft[0], lat: topLeft[1] },
    { lon: bottomLeft[0], lat: bottomLeft[1] } // Close the polygon
  ]
}

function updateSubscription() {
  if (!ws.value || !ws.value.isConnected) return

  const polygon = getMapViewPolygon()

```

```

    if (polygon.length === 0) return

    // Check if polygon has changed significantly
    if (currentSubscription && polygonsAreSimilar(currentSubscription, polygon)) {
        return
    }

    // Subscribe to new polygon (no need to unsubscribe from previous)
    ws.value.subscribeToPolygon(polygon)
    currentSubscription = polygon

    console.log('Updated subscription polygon:', polygon)
}

function polygonsAreSimilar(poly1: Point[], poly2: Point[], threshold: number = 0.01): boolean {
    if (poly1.length !== poly2.length) return false

    for (let i = 0; i < poly1.length; i++) {
        const diffLon = Math.abs(poly1[i].lon - poly2[i].lon)
        const diffLat = Math.abs(poly1[i].lat - poly2[i].lat)

        if (diffLon > threshold || diffLat > threshold) {
            return false
        }
    }

    return true
}

function setupWebSocket() {
    const url = getWebSocketUrl()

    ws.value = new TelemetryWebSocket(url, {
        onOpen: () => {
            connectionStatus.value = 'OPEN'
            connectionStatusText.value = 'Подключено'
            console.log('WebSocket connected, setting up initial subscription')

            // Initial subscription
            updateSubscription()

            // Start marker animations
            if (markerManager.value) {
                markerManager.value.startAnimation()
            }
        },

        onClose: (event) => {
            connectionStatus.value = 'CLOSED'
            connectionStatusText.value = `Отключено (код: ${event.code})`
            console.log('WebSocket closed:', event.code, event.reason)

            // Stop animations
            if (markerManager.value) {
                markerManager.value.stopAnimation()
            }
        },

        onError: (error) => {
            connectionStatus.value = 'ERROR'
            connectionStatusText.value = 'Ошибка подключения'
            console.error('WebSocket error:', error)
        },

        onReconnect: (attempt) => {
            connectionStatus.value = 'CONNECTING'
            connectionStatusText.value = `Переподключение (попытка ${attempt})`
            console.log('Reconnection attempt ${attempt}')
        },

        onMessage: (message) => {
            handleTelemetryMessage(message)
        }
    })
}

```



```

// Initialize marker manager
markerManager.value = new MarkerManager()

// Connect WebSocket
ws.value.connect()
}

function handleTelemetryMessage(message: any) {
  if (!markerManager.value || !vectorSource) return

  const deviceId = message.deviceId

  // Check if we already have a marker for this device
  const existingMarker = markerManager.value.getMarker(deviceId)

  if (existingMarker) {
    // Update existing marker
    markerManager.value.updateVehicle({
      deviceId: deviceId,
      vehicleId: message.vehicleId,
      latitude: message.latitude,
      longitude: message.longitude,
      azimuth: message.azimuth,
      packetTime: message.packetTime,
      receptionTime: message.receptionTime
    })

    // Store raw WebSocket data on the feature for popup
    existingMarker.feature.set('rawData', message)
  } else {
    // Create new marker
    const marker = markerManager.value.updateVehicle({
      deviceId: deviceId,
      vehicleId: message.vehicleId,
      latitude: message.latitude,
      longitude: message.longitude,
      azimuth: message.azimuth,
      packetTime: message.packetTime,
      receptionTime: message.receptionTime
    })

    // Store raw WebSocket data on the feature for popup
    marker.feature.set('rawData', message)

    // Double-check that the feature isn't already in the vector source
    // (race condition protection)
    const features = vectorSource.getFeatures()
    if (!features.includes(marker.feature)) {
      vectorSource.addFeature(marker.feature)
    }
  }

  // Update vehicle count
  vehicleCount.value = markerManager.value.getAllMarkers().length
}

function cleanup() {
  // Clean up WebSocket
  if (ws.value) {
    ws.value.disconnect()
    ws.value = null
  }

  // Clean up marker manager
  if (markerManager.value) {
    markerManager.value.dispose()
    markerManager.value = null
  }

  // Clean up vector source
  if (vectorSource) {
    vectorSource.clear()
    vectorSource = null
  }
}

```

```

    }

    // Clean up timers
    if (subscriptionDebounceTimer) {
      clearTimeout(subscriptionDebounceTimer)
      subscriptionDebounceTimer = null
    }

    // Clean up popup timer
    if (hidePopupTimer) {
      clearTimeout(hidePopupTimer)
      hidePopupTimer = null
    }
  }

  // Popup functions (simple approach like ZoneMap.vue)
  function cancelPopupHide() {
    if (hidePopupTimer) {
      clearTimeout(hidePopupTimer)
      hidePopupTimer = null
    }
  }

  function schedulePopupHide() {
    cancelPopupHide()
    hidePopupTimer = window.setTimeout(() => {
      if (popupVisible.value) {
        popupVisible.value = false
        popupData.value = ''
      }
      hidePopupTimer = null
    }, HIDE_POPUP_DELAY)
  }

  function closePopup() {
    popupVisible.value = false
    popupData.value = ''
  }

  function handlePointerMove(event: any) {
    if (!map.value || event.dragging) {
      return
    }

    const pixel = map.value.getEventPixel(event.originalEvent)
    const feature = map.value.forEachFeatureAtPixel(pixel, (f) => f, { hitTolerance: 10 })

    if (feature) {
      // Get the raw WebSocket data stored on the feature
      const rawData = feature.get('rawData')

      if (rawData) {
        // Format JSON for display
        popupData.value = JSON.stringify(rawData, null, 2)
      } else {
        // Fallback to feature properties
        const props = feature.getProperties()
        popupData.value = JSON.stringify(props, null, 2)
      }

      // Calculate popup position relative to map container
      const rect = mapContainer.value?.getBoundingClientRect()
      if (rect) {
        popupLeft.value = event.originalEvent.clientX - rect.left + 12
        popupTop.value = event.originalEvent.clientY - rect.top + 12
      }

      popupVisible.value = true
      cancelPopupHide()
      map.value.getTargetElement().style.cursor = 'pointer'
    } else {
      map.value.getTargetElement().style.cursor = ''
      schedulePopupHide()
    }
  }

```

```

}

onMounted(() => {
  createMap()
  setupWebSocket()
})

onBeforeUnmount(() => {
  cleanup()

  if (map.value) {
    map.value.setTarget(undefined)
    map.value = null
  }
})

// Watch for connection status changes
watch(connectionStatus, (status) => {
  console.log('Connection status changed:', status)
})
</script>

<style scoped>
.vehicles-map-container {
  width: 100%;
  height: 100%;
  position: relative;
}

.vehicles-map {
  width: 100%;
  height: 100%;
}

.connection-status {
  position: absolute;
  top: 10px;
  right: 10px;
  background: rgba(255, 255, 255, 0.9);
  border-radius: 8px;
  padding: 8px 12px;
  display: flex;
  align-items: center;
  gap: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15);
  z-index: 1000;
  font-size: 14px;
}

.status-indicator {
  width: 10px;
  height: 10px;
  border-radius: 50%;
}

.status-indicator.connecting {
  background-color: #ff9800;
  animation: pulse 1.5s infinite;
}

.status-indicator.open {
  background-color: #4caf50;
}

.status-indicator.closed {
  background-color: #f44336;
}

.status-indicator.error {
  background-color: #f44336;
  animation: pulse 1s infinite;
}

@keyframes pulse {

```

```

0% { opacity: 1; }
50% { opacity: 0.5; }
100% { opacity: 1; }
}

.vehicle-counter {
  position: absolute;
  bottom: 10px;
  left: 10px;
  background: rgba(255, 255, 255, 0.9);
  border-radius: 8px;
  padding: 6px 10px;
  font-size: 13px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15);
  z-index: 1000;
}

/* Popup styles */
.vehicle-popup {
  position: absolute;
  background: white;
  border-radius: 8px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.2);
  min-width: 300px;
  max-width: 500px;
  max-height: 400px;
  overflow: hidden;
  z-index: 1001;
  font-family: 'Courier New', monospace;
  font-size: 12px;
  line-height: 1.4;
  pointer-events: auto;
}

.popup-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px 15px;
  background: #2196f3;
  color: white;
  font-weight: bold;
  font-size: 14px;
  font-family: sans-serif;
}

.popup-title {
  flex: 1;
}

.popup-close {
  background: transparent;
  border: none;
  color: white;
  font-size: 20px;
  cursor: pointer;
  line-height: 1;
  padding: 0;
  width: 24px;
  height: 24px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 50%;
  transition: background-color 0.2s;
}

.popup-close:hover {
  background: rgba(255, 255, 255, 0.2);
}

.popup-content {
  padding: 15px;
  background: #f8f9fa;
}

```

```

    max-height: 350px;
    overflow-y: auto;
}

.json-data {
    margin: 0;
    white-space: pre-wrap;
    word-break: break-all;
    color: #333;
}

/* Dark theme for JSON syntax highlighting */
.json-data .key { color: #d32f2f; }
.json-data .string { color: #388e3c; }
.json-data .number { color: #1976d2; }
.json-data .boolean { color: #7b1fa2; }
.json-data .null { color: #616161; }
</style>

```

6. Пользовательский клиент – интерфейс для получения телеметрических данных telemetry.ts

```

export interface Point {
    lon: number;
    lat: number;
}

export interface SubscribePolygonMessage {
    type: 'subscribe_polygon';
    points: Point[];
}

export interface UnsubscribePolygonMessage {
    type: 'unsubscribe_polygon';
}

export interface TelemetryMessage {
    id: number | null;
    vehicleId: number;
    deviceId: number;
    packetTime: string;
    receptionTime: string;
    latitude: number;
    longitude: number;
    s2Cell: number;
    azimuth: number;
}

export type WebSocketMessage = SubscribePolygonMessage | UnsubscribePolygonMessage;
export type IncomingMessage = TelemetryMessage;

export interface WebSocketCallbacks {
    onOpen?: () => void;
    onClose?: (event: CloseEvent) => void;
    onError?: (error: Event) => void;
    onMessage?: (message: IncomingMessage) => void;
    onReconnect?: (attempt: number) => void;
}

// HTTP API Interfaces
export interface DeviceTelemetryRequest {
    deviceId: number;
    fromDateTime: string;
    toDateTime: string;
}

```

```

}

export type TelemetryPacket = TelemetryMessage;

// Base URL for HTTP API
const API_BASE = '/telemetry';

async function request<T>(path: string, init?: RequestInit): Promise<T> {
  const response = await fetch(`${API_BASE}${path}`, init);
  if (!response.ok) {
    const errorText = await response.text().catch(() => response.statusText);
    throw new Error(errorText || `HTTP ${response.status}`);
  }
  return response.json() as Promise<T>;
}

/**
 * Get device telemetry for a specific time period
 * POST /api/telemetry/device-telemetry
 */
export async function getDeviceTelemetry(
  payload: DeviceTelemetryRequest
): Promise<TelemetryPacket[]> {
  return request<TelemetryPacket[]>('/device-telemetry', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload)
  });
}

/**
 * Query telemetry by polygon and time interval
 * POST /api/telemetry/query-by-polygon
 */
export interface PointDto {
  latitude: number;
  longitude: number;
}

export interface PolygonTimeRequest {
  polygon: PointDto[];
  fromDateTime: string;
  toDateTime: string;
}

export interface TelemetryIntervalResponse {
  vehicleId: number;
  deviceId: number;
  fromDateTime: string;
  toDateTime: string;
  // Optional entries if provided by API
  entries?: TelemetryPacket[];
}

export async function queryTelemetryByPolygon(
  payload: PolygonTimeRequest
): Promise<TelemetryIntervalResponse> {
  return request<TelemetryIntervalResponse>('/query-by-polygon', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload)
  });
}

export class TelemetryWebSocket {
  private ws: WebSocket | null = null;
  private url: string;
  private callbacks: WebSocketCallbacks;
  private reconnectAttempts = 0;
  private maxReconnectAttempts = 10;
  private reconnectDelay = 1000;
  private isConnecting = false;
  private shouldReconnect = true;
  private reconnectTimeout: number | null = null;

```

```

constructor(url: string, callbacks: WebSocketCallbacks) {
    this.url = url;
    this.callbacks = callbacks;
}

connect(): void {
    if (this.isConnecting || this.ws?.readyState === WebSocket.OPEN) {
        return;
    }

    this.isConnecting = true;
    this.shouldReconnect = true;

    try {
        this.ws = new WebSocket(this.url);
        this.setupEventListeners();
    } catch (error) {
        console.error('Failed to create WebSocket:', error);
        this.isConnecting = false;
        this.scheduleReconnect();
    }
}

private setupEventListeners(): void {
    if (!this.ws) return;

    this.ws.onopen = () => {
        console.log('WebSocket connected to', this.url);
        this.isConnecting = false;
        this.reconnectAttempts = 0;
        this.callbacks.onOpen?.();
    };

    this.ws.onclose = (event) => {
        console.log('WebSocket disconnected:', event.code, event.reason);
        this.isConnecting = false;
        this.ws = null;
        this.callbacks.onClose?.(event);

        if (this.shouldReconnect && event.code !== 1000) {
            this.scheduleReconnect();
        }
    };

    this.ws.onerror = (error) => {
        console.error('WebSocket error:', error);
        this.isConnecting = false;
        this.callbacks.onError?.(error);
    };

    this.ws.onmessage = (event) => {
        try {
            const data = JSON.parse(event.data);
            if (this.isTelemetryMessage(data)) {
                this.callbacks.onMessage?.(data);
            } else {
                console.warn('Unknown message format:', data);
            }
        } catch (error) {
            console.error('Failed to parse WebSocket message:', error, event.data);
        }
    };
}

private isTelemetryMessage(data: any): data is TelemetryMessage {
    return (
        typeof data === 'object' &&
        data !== null &&
        typeof data.deviceId === 'number' &&
        typeof data.latitude === 'number' &&
        typeof data.longitude === 'number'
    );
}

```

```

send(message: WebSocketMessage): void {
  if (this.ws?.readyState === WebSocket.OPEN) {
    this.ws.send(JSON.stringify(message));
  } else {
    console.warn('WebSocket is not open, cannot send message:', message);
  }
}

subscribeToPolygon(points: Point[]): void {
  this.send({
    type: 'subscribe_polygon',
    points
  });
}

unsubscribeFromPolygon(): void {
  this.send({
    type: 'unsubscribe_polygon'
  });
}

private scheduleReconnect(): void {
  if (!this.shouldReconnect || this.reconnectAttempts >= this.maxReconnectAttempts) {
    console.log('Max reconnection attempts reached or reconnection disabled');
    return;
  }

  if (this.reconnectTimeout) {
    clearTimeout(this.reconnectTimeout);
  }

  this.reconnectAttempts++;
  const delay = this.reconnectDelay * Math.pow(1.5, this.reconnectAttempts - 1);

  console.log(`Scheduling reconnect attempt ${this.reconnectAttempts} in ${delay}ms`);

  this.reconnectTimeout = window.setTimeout(() => {
    this.callbacks.onReconnect?.(this.reconnectAttempts);
    this.connect();
  }, delay);
}

disconnect(): void {
  this.shouldReconnect = false;

  if (this.reconnectTimeout) {
    clearTimeout(this.reconnectTimeout);
    this.reconnectTimeout = null;
  }

  if (this.ws) {
    this.ws.onclose = null; // Prevent reconnect on manual close
    this.ws.close(1000, 'Manual disconnect');
    this.ws = null;
  }
}

get isConnected(): boolean {
  return this.ws?.readyState === WebSocket.OPEN;
}

get connectionState(): string {
  if (!this.ws) return 'DISCONNECTED';
  switch (this.ws.readyState) {
    case WebSocket.CONNECTING: return 'CONNECTING';
    case WebSocket.OPEN: return 'OPEN';
    case WebSocket.CLOSING: return 'CLOSING';
    case WebSocket.CLOSED: return 'CLOSED';
    default: return 'UNKNOWN';
  }
}
}

```



```
// Helper function to create WebSocket URL based on current environment
export function getWebSocketUrl(): string {
  const protocol = window.location.protocol === 'https:' ? 'wss:' : 'ws:';
  const host = window.location.host;
  return `${protocol}://${host}/ws-telemetry/telemetry`;
}

// Default export for convenience
export default TelemetryWebSocket;
```